RISK 231: R

Himchan Jeong, FSA

Department of Statistics & Actuarial Science
Simon Fraser University

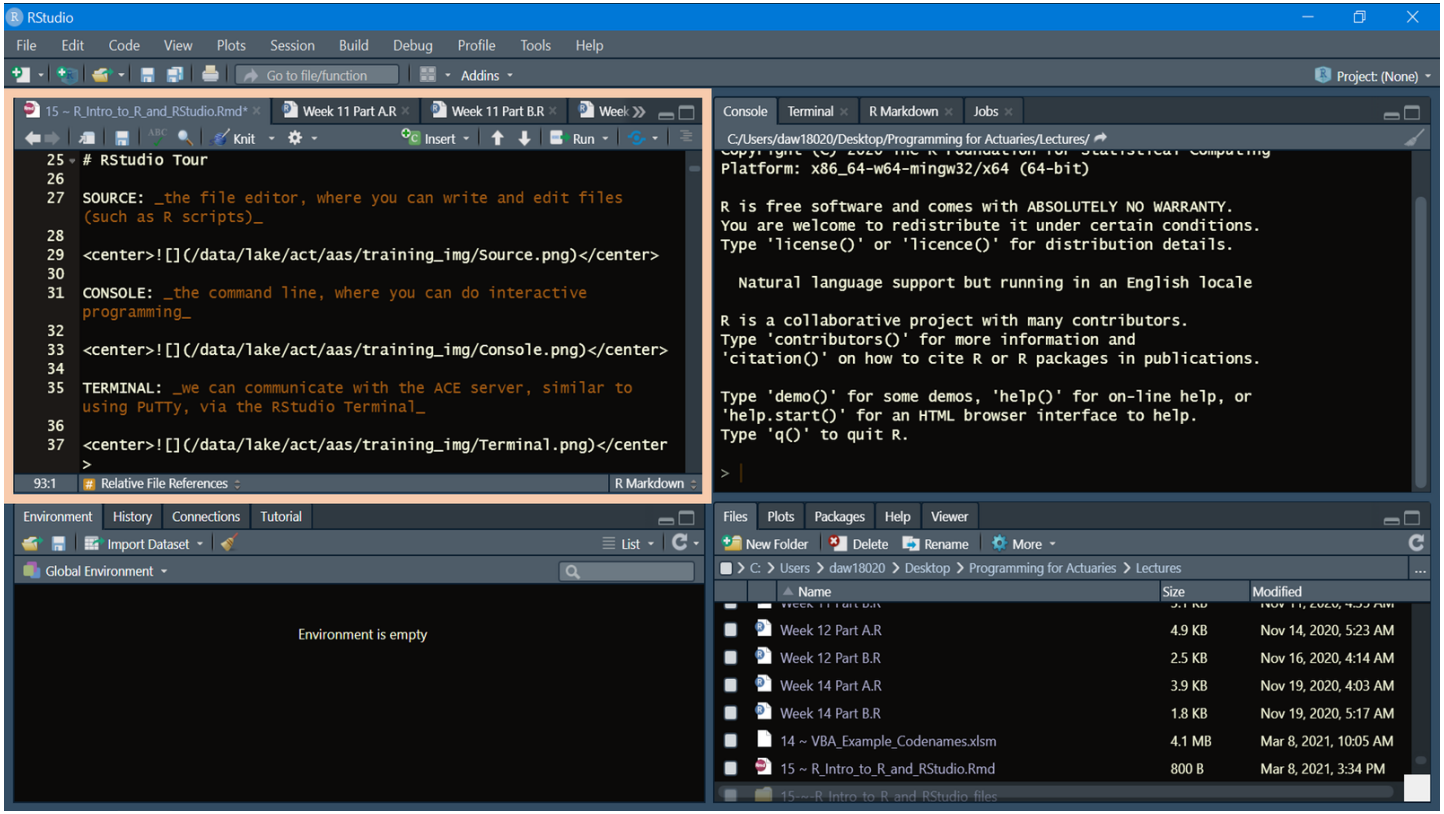
Introduction to R and RStudio

R Studio

- R and RStudio will need to be downloaded onto your computer:
  - R: https://mirror.las.iastate.edu/CRAN/
  - RStudio: https://rstudio.com/products/rstudio/download
- RStudio desktop is freeware, while RStudio Server is licensed by companies to provide the same features for employees but connected to servers.
- RStudio on your desktop uses the CPU and memory installed on your local machine.
- When using RStudio Server, employees are actually doing the 'work' remotely – RStudio is just an interface to the company's server, where the code is being executed and computations being done.

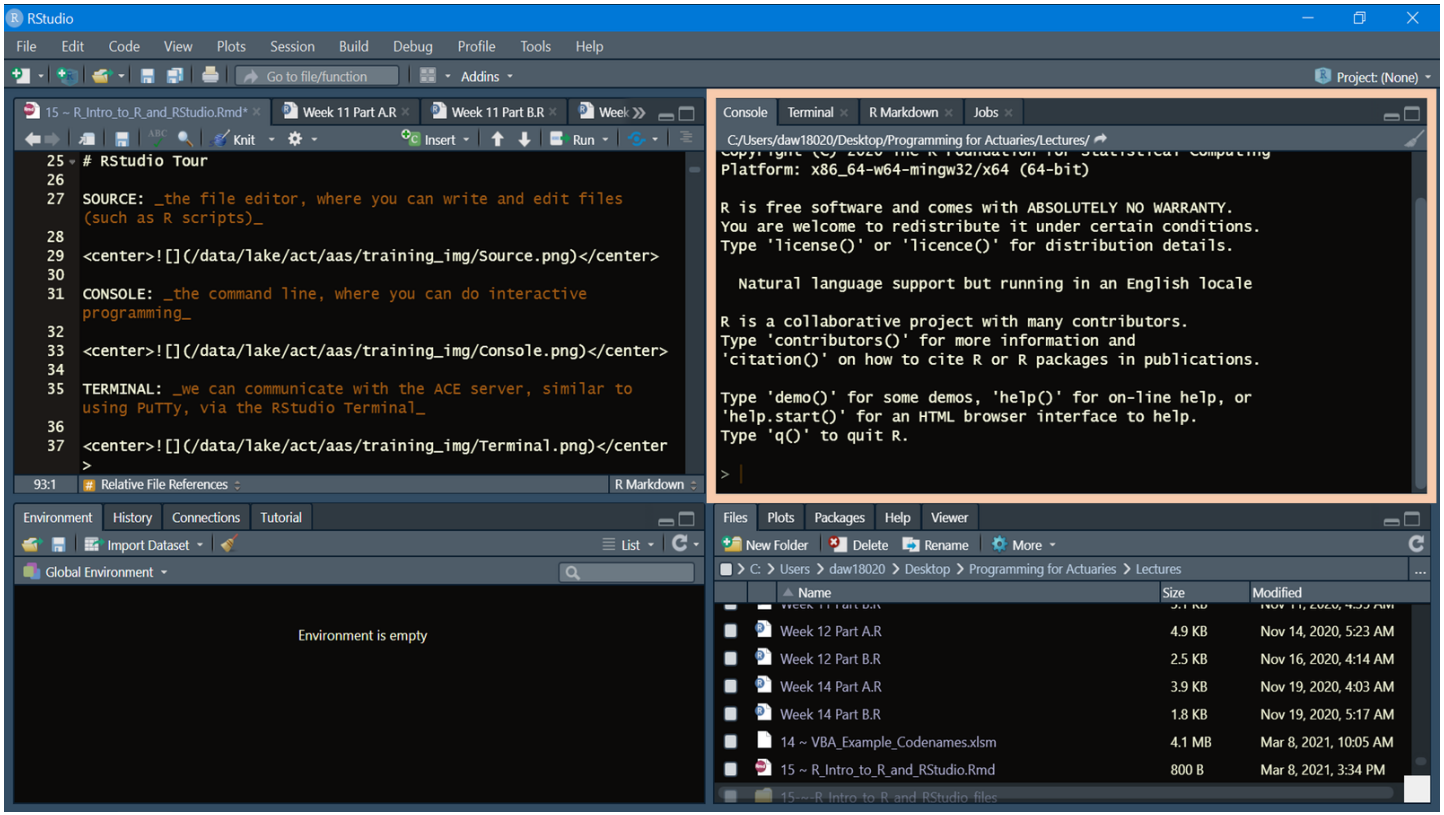
RStudio Tour

- SOURCE: *the file editor, where you can write and edit files (such as R scripts)*



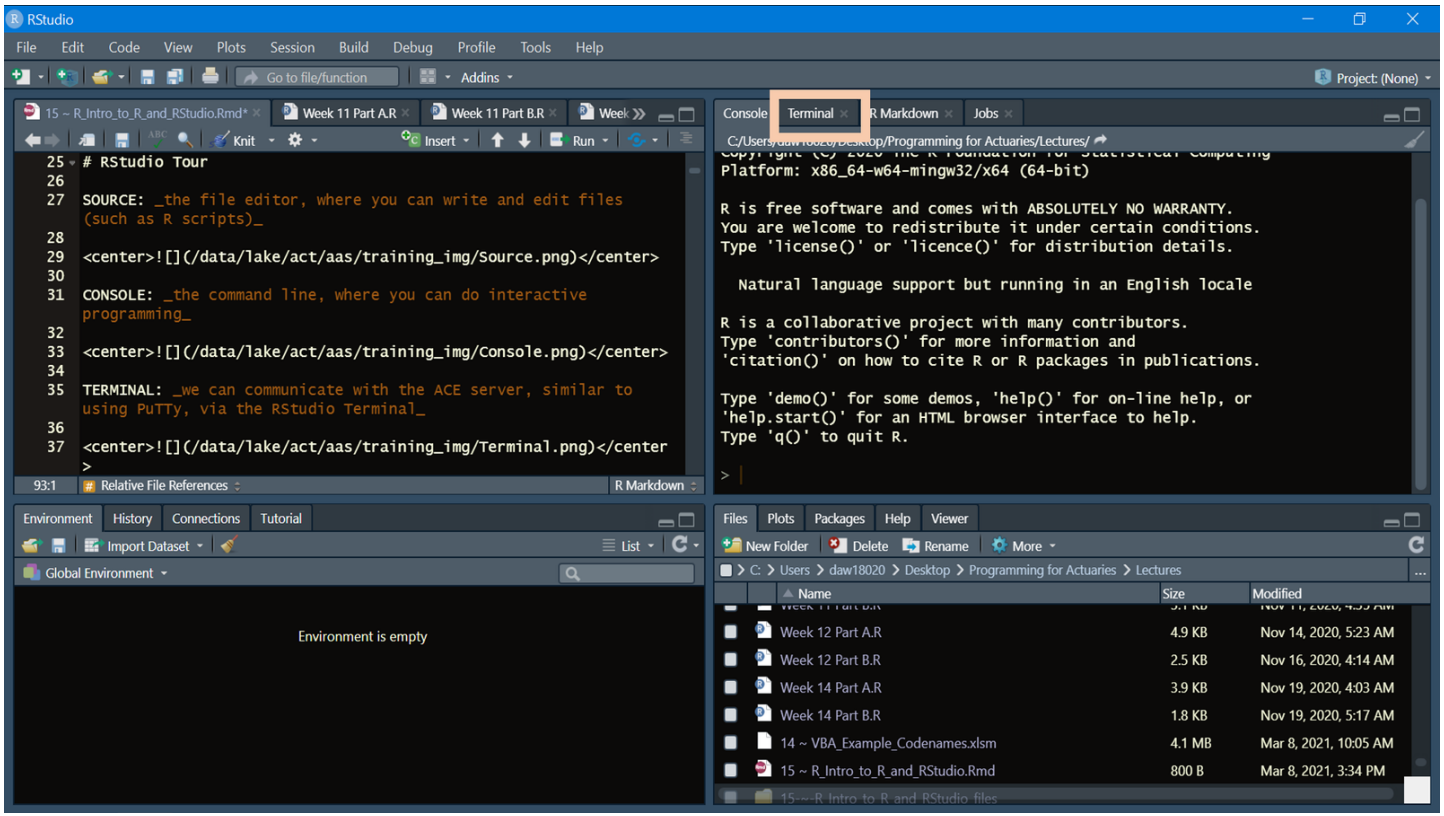
RStudio Tour (cont.)

- CONSOLE: *the command line, where you can do interactive programming*



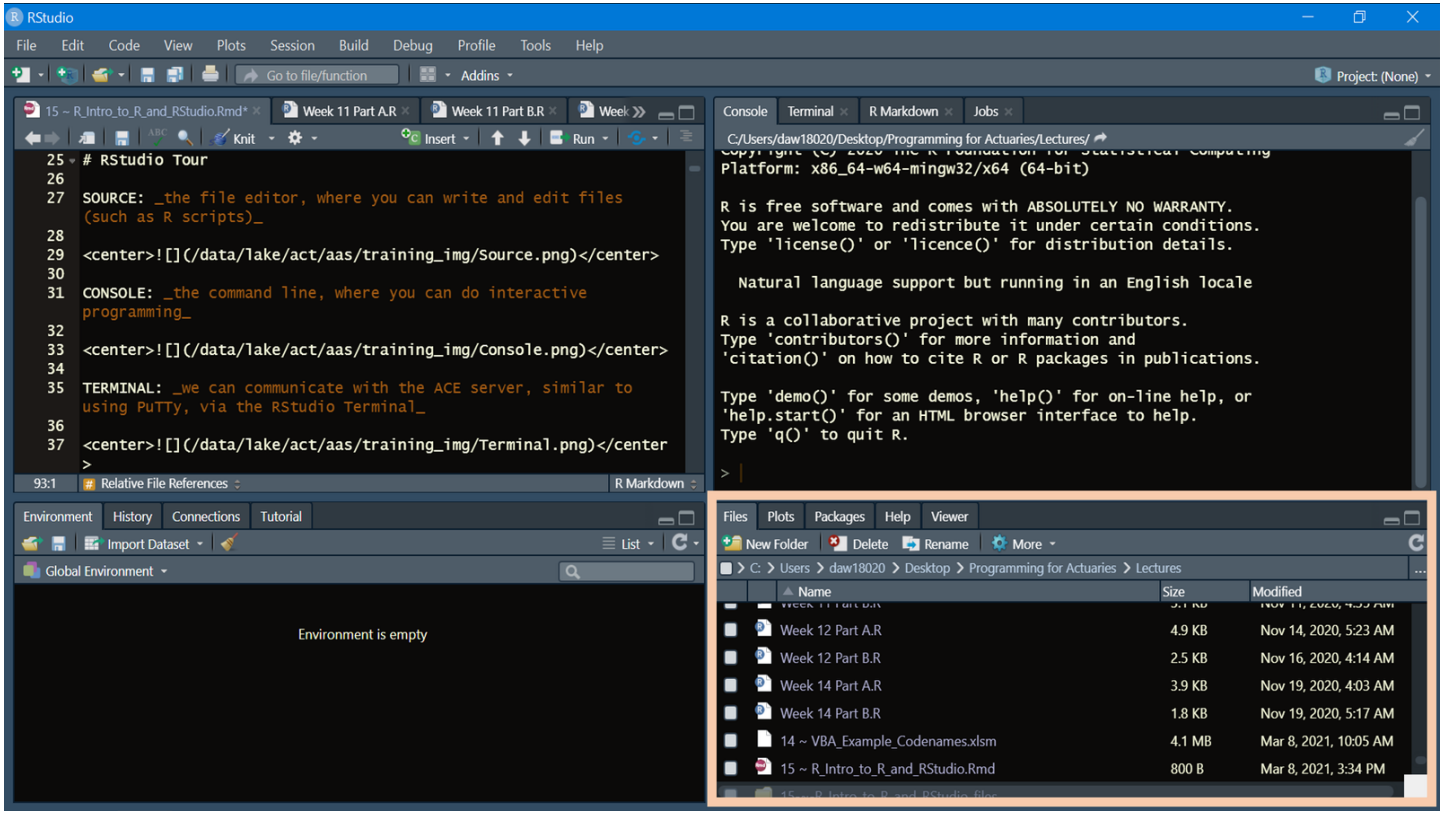
RStudio Tour (cont.)

- TERMINAL: *employees can communicate with the company server via the RStudio Terminal – not needed for this class*



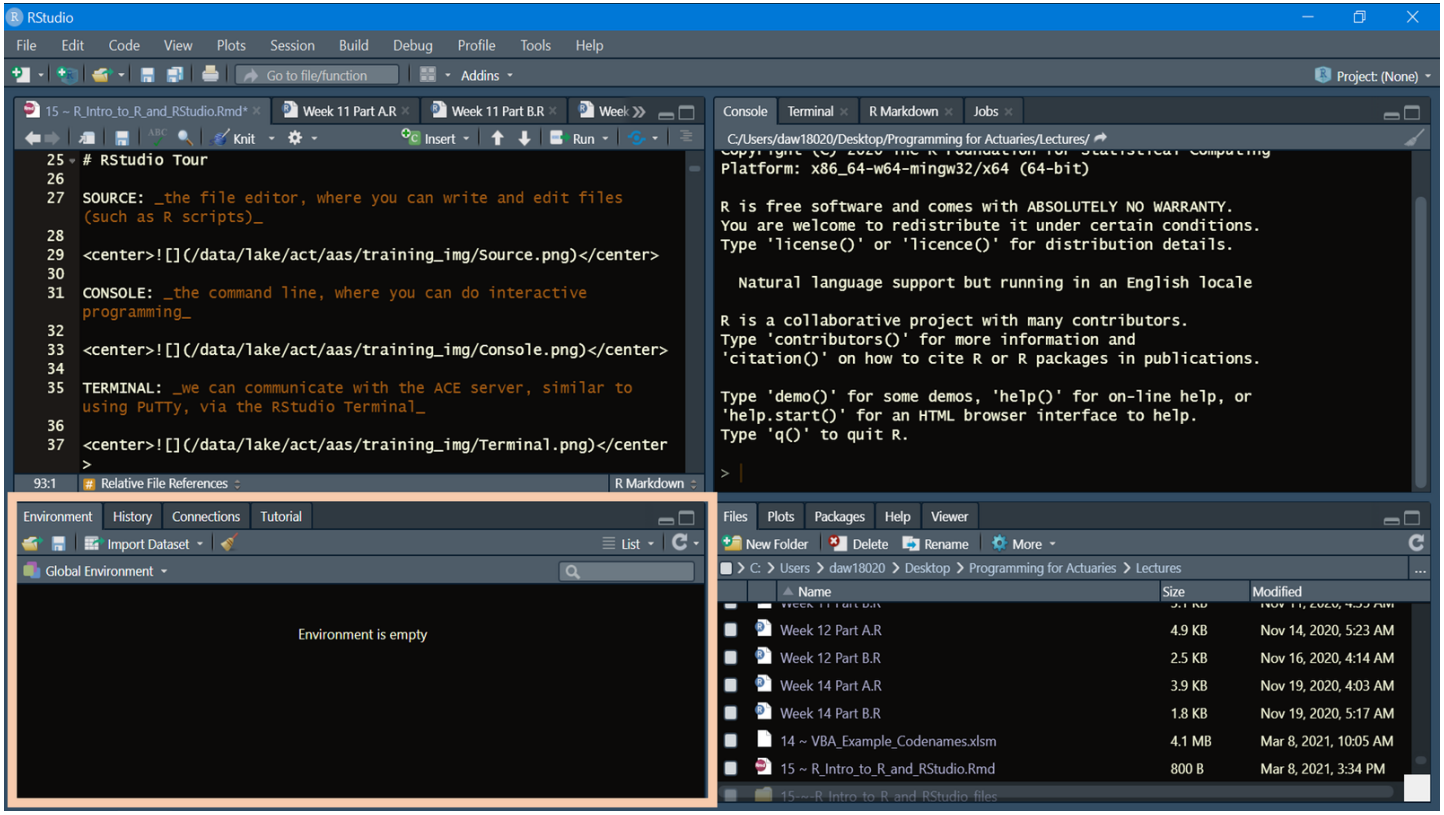
RStudio Tour (cont.)

- VIEWER: *a file browser, plot viewer, package list, and documentation viewer*



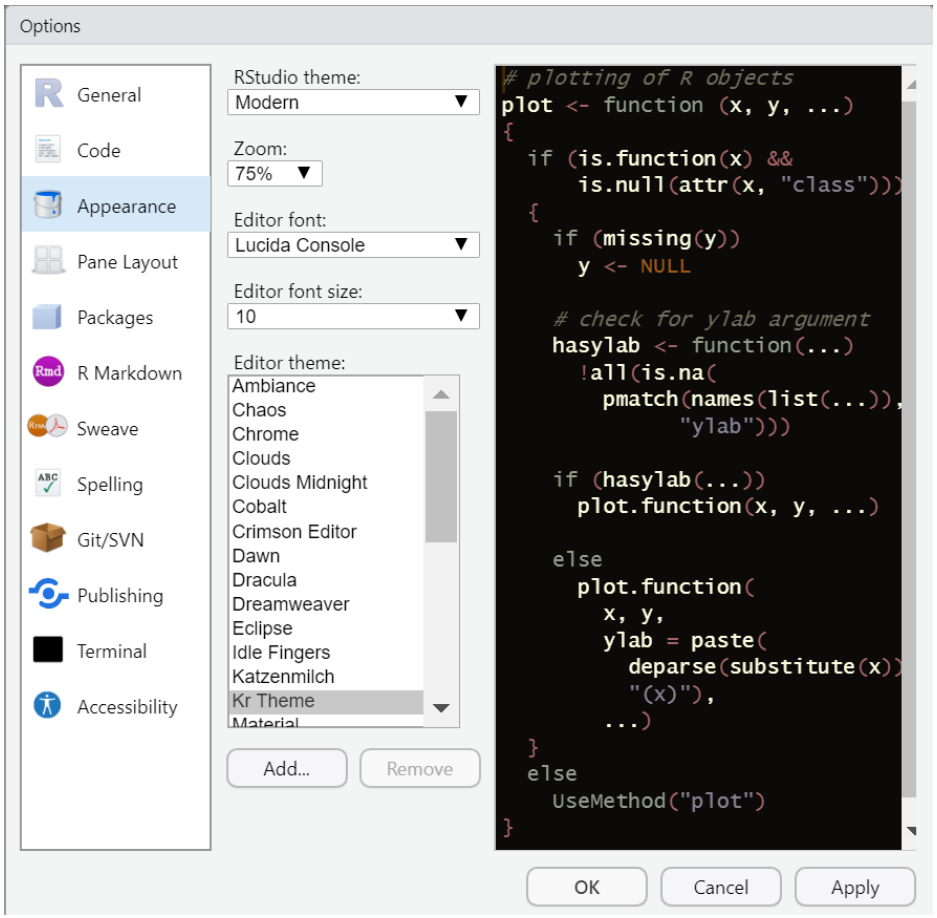
RStudio Tour (cont.)

- ENVIRONMENT: *shows the objects loaded into memory (e.g. data, variables, & functions)*



RStudio Setup

- Go to Tools → Global Options
- It may be fun to setup your appearance:



Relative File References

- When writing your code, rather than writing out the long file paths each time we need to reference something, it may be helpful to have a starting place, so to speak.
- A `working directory` allows you to set a root folder for all the file paths within your code.
- For example, if you set your project folder as your working directory, then any code you write within the project will only need to be something like `functions/my_function.R` instead of `C:/Users/ssaul/Desktop/Teaching/ACMA 231/Fall 2024/my_function.R`.
- This is called a `relative file reference`.

Relative File References (cont.)

- When beginning a project, you can set your working directory by writing code like this:

```
setwd("C:/Users/ssaul/Desktop/Teaching/ACMA 231")
```

- Or even better, you can create a `.Rproj`. This file allows you to do things like:

1. Save your workspace for quicker access to environment objects
2. Load your workspace and define a working directory on opening
3. More easily incorporate version control
4. Set file permissions for collaborators

Relative File References: Creating a `.RProj` File

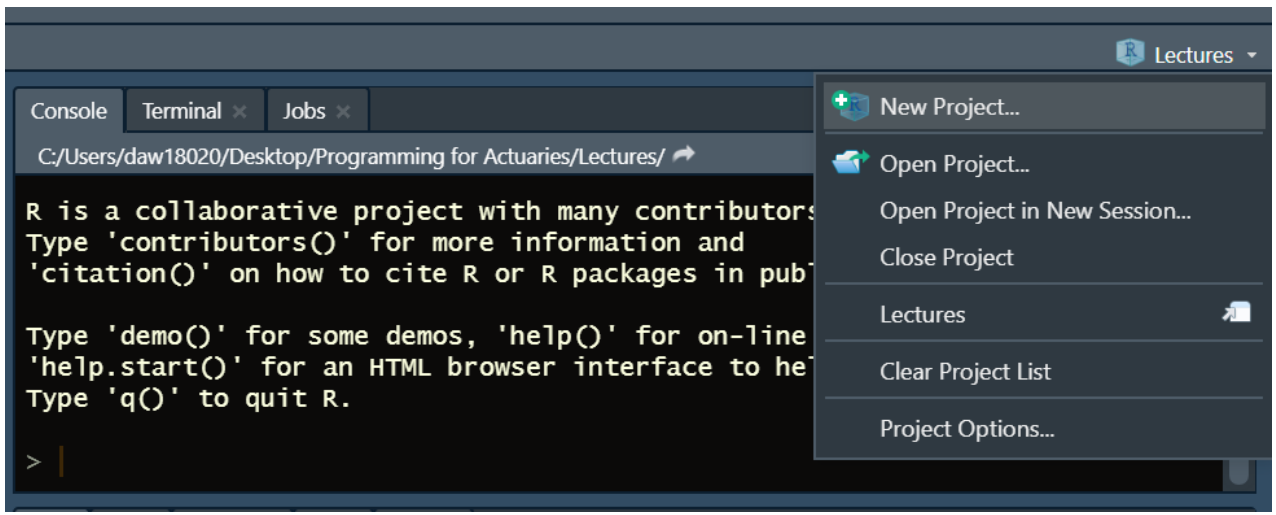
- Let's practice creating a `.RProj` file:
- *Navigate to the folder where you are keeping your RISK 231 notes. Once inside, create a new folder called `Intro to R`. Once inside the new folder click on this icon in the top left corner of your RStudio:*



- *Select `Existing Directory` and enter the file path to your new folder.*
- *Click `Create Project` and your `.RProj` file will be created in the new folder!*

Relative File References: Switching Projects

- *Not only is this handy for setting your working directory, but if you are working on several projects, you can easily close one and open another, setting it automatically as your new working directory by using the Project Dropdown feature in the top right corner of RStudio:*



Main R File Types

- We already discussed `.Rproj` files and how useful they are...
- A `.R` file is a saved set of code which you can re-run at any time.
- A `.Rmd` file is an RMarkdown, which is a simple formatting syntax for authoring HTML pages, PDF documents, presentations, and MS Word documents.
  - RMarkdown is powerful because you can incorporate plain text/documentation, R code, and R code output all in one document.
  - In fact, the original HTML version of these lecture notes is a form of RMarkdown called Slidy.

Creating a .R (R Script)

- Let's try it out!
- *Click on `File` in the top left of RStudio and select `New File`.*
- *You can see from the list that you can select a .R file (R Script), an RMarkdown file, and many others. Let's create a .R file.*
- *After clicking on `R Script`, a new window should have opened in your Source window, labeled "Untitled".*
- *Click the save button on the top left of the window and browse to your new folder and name the .R file "R_Intro_Script".*

Creating a .R (R Script) (cont.)

- Once your file is saved, type the following into the Source window:

```
# Chunk 1
2 + 1

# Chunk 2
21:50

# Chunk 3
a <- 2 + 1
b <- 5 - 2
c <- a + b
```

```
# Chunk 4
c
print(c)

# Chunk 5
d <- "Hello World"
class(d)

# Chunk 6
e <- TRUE
class(e)
```

Creating a .R (R Script): Run vs. Source

- Place your cursor at the top of the script and then click on the `Run` button at the top-right of the Source window. . . or you can use `CTRL+Enter`.
  - Notice that the `Run` button will run the code one chunk at a time. If you keep clicking it, it will move on to the next chunk and then the next.
- Place your cursor at the top of the script and then click the `Source` button in the top right corner of the Source window. . . or you can use `CTRL+Shift+Enter`.
  - The `Source` button will run your entire script.

Creating a .R (R Script): What We Learned

We can learn a few things from the above code and its output:

- We can comment in R by using the `#` sign. This will allow us to include helpful documentation in our code without R trying to run it.
- `:` includes all the numbers from a given number to another.
- In R, we will use `<-` as assignment operator.
  - `=` will produce the same result, but it is not recommended because it can be easily confused with `==` which is used for equalities and comparisons.
  - NOTE: The shortcut for `<-` is `Alt+minus`.

Creating a .R (R Script): What We Learned (cont.)

- We can display our assigned variables by either writing them alone on a line or by using the `print()` function, which will explicitly show the results in the console.
- Notice in the Environment window that `a`, `b`, and `c` appear. These variables are now in your environment and can be used in later code.
- If desired, you can clear your environment by clicking on the broom button: 
- We can create character and boolean variables as well. You can use the `class()` function to check a variable's type.

Creating a .Rmd (RMarkdown)

- Let's try it out!
- *Click on `File` in the top left of RStudio and select `New File`.*
- *Select `RMarkdown`. Usually, we want to create an HTML document, so we generally leave the defaults as-is, but RMarkdown has the ability to produce PDFs or Word docs. Leave it on HTML, title it any way you'd like, and click `OK`.*
- *After clicking on `OK`, a new window should have opened in your Source window, labeled "Untitled".*
- *Click the save button on the top left of the window and browse to your new folder and name the .Rmd file "R_Intro_Markdown".*

Creating a .Rmd (RMarkdown) (cont.)

- Note that your new .Rmd file is not a blank document – you get a very basic example RMarkdown file all filled out for you as a starting point!
- We selected an HTML document. We can see what our RMarkdown looks like by pressing the `Knit` button in the top left corner of the Source window. Try it out.
- Pretty cool, right? RMarkdown can display your notes, it can insert links, it can display and run code, it can show exhibits, and much more!
- Notice in today's folder that there is an HTML version of your RMarkdown in there. Every time you `Knit` an RMarkdown, it saves your .Rmd file and creates this HTML, which you can open directly.

Working in RMarkdown

- Find a helpful RMarkdown cheat sheet here: RMarkdown Cheat Sheet
- Take a look at your `R_Intro_Markdown.Rmd` file. Let's walk through some of the sections of the template:
  - The top portion is called the "YAML Header". There are A TON of things you can do with it to adjust the file with output specification, themes, adding table of contents, and so on.
  - The next section *(lines 6–8)* is usually left alone.
  - In the text sections, you can write your documentation with all sorts of formatting possibilities. For example, you can create headers in your document using the # command.
  - You can find WAY more formatting choices by looking at the cheat sheet.

Working in RMarkdown (cont.)

- The next section shows how you can embed R code right into your documentation and decide if it will run or not.
  - The R code is placed between two sets of ``` with the `{r}` statement to start it off. This statement allows for many options for your R code.
- *On the second line within the R code section, replace* `summary(cars)` *with the code we used earlier (Chunks 1–6).*
- *After replacing the code chunk, go ahead and* `Knit` *your file. What happens?* Cool, right?
- Finally, we can add exhibits, plots, and a plethora of great displays into our RMarkdowns.

An Example R Markdown Document

About This Example

- This section reproduces the knitted HTML output of the default RMarkdown template (`R_Intro_Markdown.Rmd`) created in the previous section, with a few equations added.
- Compare it with your own knitted file: the text, the R code chunk, its output, and the plot all come from the same `.Rmd` file.

R Markdown

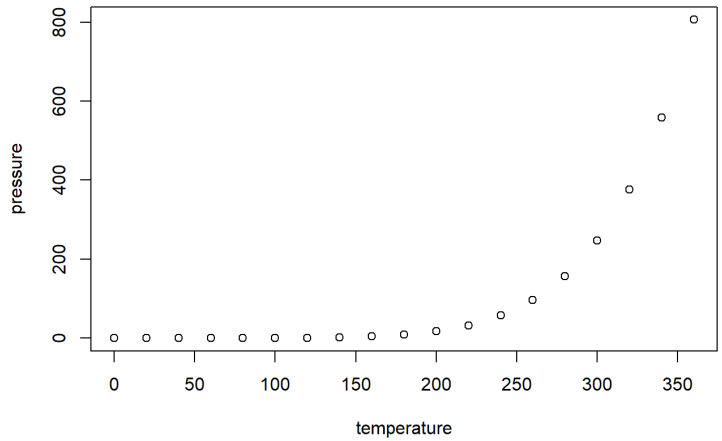
- This is an R Markdown document. Markdown is a simple formatting syntax for authoring HTML, PDF, and MS Word documents. For more details on using R Markdown see http://rmarkdown.rstudio.com.
- When you click the **Knit** button a document will be generated that includes both content as well as the output of any embedded R code chunks within the document. You can embed an R code chunk like this:

```
summary(cars)
```

```
     speed           dist       
 Min.   : 4.0   Min.   :  2.00  
 1st Qu.:12.0   1st Qu.: 26.00  
 Median :15.0   Median : 36.00  
 Mean   :15.4   Mean   : 42.98  
 3rd Qu.:19.0   3rd Qu.: 56.00  
 Max.   :25.0   Max.   :120.00  
```

Including Plots

- You can also embed plots, for example:



- Note that the `echo = FALSE` parameter was added to the code chunk to prevent printing of the R code that generated the plot.
- You can also write equations (LaTeX syntax), e.g.

$$y_i = X_i\beta + \epsilon_i, \qquad X = UDV^T$$

Variables, Vectors, Matrices, and Lists

Operators & Operator-like Functions

- **Variable Assignment** : The preferred method is using `<-` but `=` also works *(Note: When you are in the R Coding environment, you can type `Alt + -` to enter `<-`)*

```
x <- 5
y = 11
```

- **Arithmetic Operators** : `+ - / *`

```
x + y
x / y
x * y
```

```
[1] 16
[1] 0.4545455
[1] 55
```

Operators & Operator-like Functions (cont.)

- **Sequences** : Use `:` to create a sequence of numbers.
- **Equality Comparison** : use `==` to test whether items are equal.
- **Inequality Comparison** : Use `< > <= >=` to test inequalities.
- **Negation** : Use `!` to negate a logical TRUE / FALSE value

```
1:10
5 == 6
5 <= 6
5 != 6
```

```
 [1]  1  2  3  4  5  6  7  8  9 10
[1] FALSE
[1] TRUE
[1] TRUE
```

Operators & Operator-like Functions (cont.)

- **AND** : Use `&`
- **OR** : Use `|`
- **To see if an element is contained in a vector/list** : Use `%in%`
- **TRUE/FALSE reserved words** : `TRUE, FALSE, T, F`

```
(5 == 5) & (4 < 3)
(5 == 5) | (4 < 3)
2 %in% 1:3
TRUE == T
FALSE == F
```

```
[1] FALSE
[1] TRUE
[1] TRUE
[1] TRUE
[1] TRUE
```

Operators & Operator-like Functions

Practice

- Do the following:

1. Assign the following variables: 14 to a, 122 to b, 201 to c, and 5 to e.

```
a <- 14
b <- 122
c <- 201
e <- 5
```

2. Assign the sum of a, b, and c to d

```
d <- a + b + c
```

3. Assign the product of b times c to f

```
f <- b * c
```

Operators & Operator-like Functions (cont.)

Practice

4. Assign the sequence of numbers from 5 to 10 to g

```
g <- 5:10
```

5. Determine which values in g are equal to 6

```
g == 6
```

```
[1] FALSE  TRUE FALSE FALSE FALSE FALSE
```

6. Assign the sequence of numbers from 7 to 8 to h

```
h <- 7:8
```

Operators & Operator-like Functions (cont.)

Practice

7. Determine which values in g are greater than h

```
g > h
```

```
[1] FALSE FALSE FALSE FALSE  TRUE  TRUE
```

8. Negate the results of Q5

```
g != 6
```

```
[1]  TRUE FALSE  TRUE  TRUE  TRUE  TRUE
```

9. Negate the results of Q7

```
!(g > h)
```

```
[1]  TRUE  TRUE  TRUE  TRUE FALSE FALSE
```

Operators & Operator-like Functions (cont.)

Practice

10. Is a less than b and is b less than c?

```
a < b & b < c
```

```
[1] TRUE
```

11. Is a greater than d or f?

```
a > d | a > f
```

```
[1] FALSE
```

12. Is 5 in a,b,c,d,e, or f?

```
5 %in% c(a,b,c,d,e,f)
```

```
[1] TRUE
```

R Classes

- There are 4 R classes:

1. **Vector** – *a one-dimensional data structure, where the elements are all of the same type*
2. **Matrix** – *a two-dimensional table of vectors, where the elements are all of the same type*
3. **List** – *a one-dimensional vector that can contain objects of different types*
4. **Dataframe** – *a two-dimensional table of vectors, that can contain data of different types*

Vectors

- *A one-dimensional data structure, where the elements are all of the same type*

There are 3 main ways to create a vector:

- First, you could use `c()` function (c stands for "combine"):

```
v1 <- c(1, 2, 3, 4, 5)
v2 <- c("a", "b", "c")

# Not recommended
v3 <- c(1, "a")
```

```
v1
```

```
[1] 1 2 3 4 5
```

```
v2
```

```
[1] "a" "b" "c"
```

Vectors (cont.)

```
v3
```

```
[1] "1" "a"
```

- Second, you could use `seq(from, to, by)`:

```
v4 <- seq(1, 10, 2)
v5 <- seq(0, 1, 0.25)
```

```
v4
```

```
[1] 1 3 5 7 9
```

```
v5
```

```
[1] 0.00 0.25 0.50 0.75 1.00
```

- Third, you could use `rep(elements, times)` to create a vector of repeated elements:

```
awesome <- rep("Boom", 4)
awesomer <- rep(c("three", "blind", "mice"), 2)
awesomest <- c(rep(NA,16),"Batman!")
```

Vectors (cont.)

```
awesome
```

```
[1] "Boom" "Boom" "Boom" "Boom"
```

```
awesomer
```

```
[1] "three" "blind" "mice"  "three" "blind" "mice"
```

```
awesomest
```

```
 [1] NA        NA        NA        NA        NA        NA        NA
 [8] NA        NA        NA        NA        NA        NA        NA
[15] NA        NA        "Batman!"
```

Vectors

- `is.vector()` returns TRUE if an object is a vector:

```
is.vector(v5)
```

```
[1] TRUE
```

- Use the `length()` function to obtain the length of a vector:

```
length(v1)
```

```
[1] 5
```

```
length(v2)
```

```
[1] 3
```

Vectors (cont.)

- We can do operations using vectors:

```
v1
```

```
[1] 1 2 3 4 5
```

```
v1 + 2
```

```
[1] 3 4 5 6 7
```

```
# addition between two vectors
v1
```

```
[1] 1 2 3 4 5
```

```
v4
```

```
[1] 1 3 5 7 9
```

```
v1 + v4
```

```
[1]  2  5  8 11 14
```

Vectors (cont.)

```r
# combining two vectors
v14 <- c(v1, v4)

# Adding two vectors that are not same length (Not recommended)
v6 <- c(1, 2, 3)

v7 <- c(1, 2, 3, 4, 5)

# Shorter: 1 2 3 1 2 (recycled)
# Longer:  1 2 3 4 5
# Result:  2 4 6 5 7
```

```r
v6 + v7
```

```
Warning in v6 + v7: longer object length is not a multiple of shorter
    object
length
```

```
[1] 2 4 6 5 7
```

Vectors

- Extracting a subset of a vector:

```
vec1 <- c("e", "x", "a", "m", "p", "l", "e")
```

```
vec1[1]
```

```
[1] "e"
```

```
vec1[3:5]  # extract the 3rd to 5th elements
```

```
[1] "a" "m" "p"
```

```
vec1[c(4, 7)]  # extract the 4th and 7th elements
```

```
[1] "m" "e"
```

```
vec1[-1]  # negation indexing: extract all elements except the 1st
```

```
[1] "x" "a" "m" "p" "l" "e"
```

Vectors (cont.)

```
vec1[-(3:5)]  # extract all elements except 3rd to the 5th elements
```

```
[1] "e" "p" "l" "e"
```

```
vec1[-c(4, 7)]  # extract all elements except the 4th and 7th
```

```
[1] "e" "x" "a" "p" "l"
```

- Truncation using a logical vector:

```
x <- c(1, 10, 8, 2, 6)

x[x > 4]
```

```
[1] 10  8  6
```

```
# logical vector
x > 4
```

```
[1] FALSE  TRUE  TRUE FALSE  TRUE
```

Vectors (cont.)

- Override specific elements in a vector:

```
v4 <- rep(NA, 20)
v4
```

```
 [1] NA NA NA NA NA NA NA NA NA NA NA NA NA NA NA NA NA NA NA NA
```

```
v4[1:10] <- 1
v4
```

```
 [1]  1  1  1  1  1  1  1  1  1  1 NA NA NA NA NA NA NA NA NA NA
```

```
v4[11:20] <- 2
v4
```

```
 [1] 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2
```

Vectors

Practice

- Create the following vectors:

1. Values from 55 to 75.

```
seq(55,75)
```

```
 [1] 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75
```

2. 'You', 'are', 'the', 'best!' repeated 3 times.

```
rep(c('You', 'are',  'the', 'best!'),3)
```

```
 [1] "You"   "are"   "the"   "best!" "You"   "are"   "the"   "best!"
 [9] "You"   "are"   "the"   "best!"
```

Vectors (cont.)

Practice

3. Even numbers from 20 to 84.

```
seq(from = 20, to = 84, by = 2)
```

```
 [1] 20 22 24 26 28 30 32 34 36 38 40 42 44 46 48 50 52 54 56 58 60 62
    64 66 68
[26] 70 72 74 76 78 80 82 84
```

4. What is the length of the vector created in Q3?

```
length(seq(from = 20, to = 84, by = 2))
```

```
[1] 33
```

5. Assign 34 to y. Is y a vector?

```
y <- 34
is.vector(y)
```

```
[1] TRUE
```

Matrix

- *A two-dimensional table of vectors, where the elements are all of the same type*
- To create a matrix in R, we can use `matrix(data, [nrow], [ncol], [byrow])`:

```
A <- matrix(1:8)
```

```
A
```

```
     [,1]
[1,]    1
[2,]    2
[3,]    3
[4,]    4
[5,]    5
[6,]    6
[7,]    7
[8,]    8
```

```
A_4x2 <- matrix(1:8, ncol = 2)
A_4x2
```

```
     [,1] [,2]
[1,]    1    5
[2,]    2    6
[3,]    3    7
[4,]    4    8
```

Matrix (cont.)

```
A_4x2_byrow <- matrix(1:8, ncol =
    2,
                      byrow = TRUE)
A_4x2_byrow
```

```
     [,1] [,2]
[1,]    1    2
[2,]    3    4
[3,]    5    6
[4,]    7    8
```

```
# the data can also be a vector
B <- matrix(c(2, 3, 1, 4),
            nrow = 2)
B
```

```
     [,1] [,2]
[1,]    2    1
[2,]    3    4
```

Matrix (cont.)

- Looking at matrix attributes: `nrow()`, `ncol()`, `dim()`:

```
nrow(A_4x2)
```

```
[1] 4
```

```
ncol(A_4x2)
```

```
[1] 2
```

```
dim(A_4x2)
```

```
[1] 4 2
```

```
dim(A_4x2)[1]
```

```
[1] 4
```

Matrix

- Extract elements from a matrix using the indexing feature:

```
B
```

```
     [,1] [,2]
[1,]    2    1
[2,]    3    4
```

```
B[2, 2]  # second row & second column
```

```
[1] 4
```

```
B[1, ]  # first row & all columns
```

```
[1] 2 1
```

```
B[, 2]  # all rows & second column
```

```
[1] 1 4
```

Matrix (cont.)

- Basic operations using matrices:

```
C <- matrix(1:6, ncol = 2)
D <- matrix(7:12, ncol = 2)

C
```

```
     [,1] [,2]
[1,]    1    4
[2,]    2    5
[3,]    3    6
```

```
D
```

```
     [,1] [,2]
[1,]    7   10
[2,]    8   11
[3,]    9   12
```

```
C + D
```

```
     [,1] [,2]
[1,]    8   14
[2,]   10   16
[3,]   12   18
```

Matrix (cont.)

```
C * D  # element-wise multiplication (NOT matrix multiplication)
```

```
     [,1] [,2]
[1,]    7   40
[2,]   16   55
[3,]   27   72
```

```
C == D  # to check equivalence, you must use "=="
```

```
      [,1]  [,2]
[1,] FALSE FALSE
[2,] FALSE FALSE
[3,] FALSE FALSE
```

Matrix (cont.)

```
t(C)  # transpose
```

```
     [,1] [,2] [,3]
[1,]    1    2    3
[2,]    4    5    6
```

```
C %*% t(D)  # matrix multiplication
```

```
     [,1] [,2] [,3]
[1,]   47   52   57
[2,]   64   71   78
[3,]   81   90   99
```

```
solve(B)  # the inverse matrix
```

```
     [,1] [,2]
[1,]  0.8 -0.2
[2,] -0.6  0.4
```

Matrix

- Use `cbind()` to join by cols and `rbind()` to join by rows:

```
v1 <- c(1, 3, 5)
v2 <- c(2, 4, 6)

mat_r <- rbind(v1, v2) # bind these 2 vectors in a form of row vectors
```

```
mat_r
```

```
   [,1] [,2] [,3]
v1    1    3    5
v2    2    4    6
```

```
mat_c <- cbind(v1, v2) # bind these 2 vectors in a form of col vectors
```

Matrix (cont.)

```
mat_c
```

```
     v1 v2
[1,]  1  2
[2,]  3  4
[3,]  5  6
```

- The matrix will always be of one type. Therefore, if you are attempting to make a matrix with vectors of different types, R will coerce all the data to be of the most general type. For example:

```
a <- c(1,2,3)
b <- c(4,5,6)
c <- c("a","b","c")

my_matrix <- as.matrix(cbind(a,b,c)) # coerces all values to characters
```

Matrix (cont.)

```
my_matrix
```

```
     a   b   c
[1,] "1" "4" "a"
[2,] "2" "5" "b"
[3,] "3" "6" "c"
```

```
my_matrix2 <- as.matrix(cbind(a,b)) # keeps as numeric
```

```
my_matrix2
```

```
     a b
[1,] 1 4
[2,] 2 5
[3,] 3 6
```

List

- *A one-dimensional vector that can contain objects of different types*
- Use `list()` to create a collection of components:

```
p <- "a list"
q <- 12:16
r <- matrix(1:9, ncol = 3)

# Putting objects p, q, and r into a list
my_list <- list(p, q, r)
```

List (cont.)

```
my_list
```

```
[[1]]
[1] "a list"

[[2]]
[1] 12 13 14 15 16

[[3]]
     [,1] [,2] [,3]
[1,]    1    4    7
[2,]    2    5    8
[3,]    3    6    9
```

- Subset a list using square brackets:

```
my_list[[1]]
```

```
[1] "a list"
```

List (cont.)

```
# extract a certain elements in a component
my_list[[2]][2] # extract the 2nd element in the 2nd component
```

```
[1] 13
```

- Can also access subset by the names of component:

```
# Add names to components in a list (using "names()")
names(my_list) <- c("p", "q", "r")
my_list
```

```
$p
[1] "a list"

$q
[1] 12 13 14 15 16

$r
     [,1] [,2] [,3]
[1,]    1    4    7
[2,]    2    5    8
[3,]    3    6    9
```

List (cont.)

```
# Access components by name in
    list
my_list$r
```

```
     [,1] [,2] [,3]
[1,]    1    4    7
[2,]    2    5    8
[3,]    3    6    9
```

```
my_list$r[3, 3]
```

```
[1] 9
```

Data Frames, Factors & Tables, and File I/O

Dataframe

- *A two-dimensional table of vectors, that can contain data of different types*
- To create a data frame, use `data.frame()`:

```
x <- 1:5
y <- c("M", "F", "M", "M", "F")
z <- c("one", "two", "three", "four", "five")

my_df <- data.frame(x, y, z)
my_df
```

```
  x y     z
1 1 M   one
2 2 F   two
3 3 M three
4 4 M  four
5 5 F  five
```

Dataframe: Attributes

```
my_df
```

```
  x y     z
1 1 M   one
2 2 F   two
3 3 M three
4 4 M  four
5 5 F  five
```

- Attributes *(similar to matrix)*:

```
nrow(my_df)  # number of observations
```

```
[1] 5
```

```
ncol(my_df)  # number of variables
```

```
[1] 3
```

Dataframe: Attributes (cont.)

```
dim(my_df)  # dimensions of the dataframe
```

```
[1] 5 3
```

- Other helpful attribute functions:
  - `head(df, n)` will look at the first *n* observations of a dataframe
  - `tail(df, n)` will look at the last *n* observations of a dataframe

```
head(my_df, 3)
```

```
  x y     z
1 1 M   one
2 2 F   two
3 3 M three
```

```
tail(my_df, 3)
```

```
  x y     z
3 3 M three
4 4 M  four
5 5 F  five
```

Dataframe: Subsetting

```
my_df
```

```
  x y     z
1 1 M   one
2 2 F   two
3 3 M three
4 4 M  four
5 5 F  five
```

- Subsetting a data frame:
- First, using a pair of indices, like a matrix:

```
my_df[1, 2]  # extract the element in the 1st row and 2nd col
```

```
[1] "M"
```

```
my_df[, 1]  # extract only the 1st col
```

```
[1] 1 2 3 4 5
```

Dataframe: Subsetting (cont.)

```
my_df[3, 2:3]  # 3rd row, col 2 through 3
```

```
  y     z
3 M three
```

```
my_df[c(1, nrow(my_df)), ]  # extract the 1st and the last obs
```

```
  x y    z
1 1 M  one
5 5 F five
```

- Second, by name:

```
names(my_df)  # use names() to look at the names
```

```
[1] "x" "y" "z"
```

Dataframe: Subsetting (cont.)

```
my_df$x  # access variable x
```

```
[1] 1 2 3 4 5
```

```
my_df$x[4]  # access the 4th element in variable x
```

```
[1] 4
```

```
mean(my_df$x)  # calculate the mean of variable x
```

```
[1] 3
```

Miscellaneous Functions for Text Manipulation

- `paste()` joins strings; by default, `paste()` will use `sep = " "`

```
paste("a", "b")
```

```
[1] "a b"
```

```
paste("a", "b", sep = "")
```

```
[1] "ab"
```

```
paste0("a", "b")  # a special version of paste() assuming sep = ""
```

```
[1] "ab"
```

Miscellaneous Functions for Text Manipulation (cont.)

- Assigning two vectors:

```
c1 <- c("a", "b")
c2 <- c("c", "d")
```

- Difference between `sep` and `collapse`:
- `paste0(c1, c2)` vs `paste0(c1, c2, collapse = "+")`:

```
paste0(c1, c2)
```

```
[1] "ac" "bd"
```

```
paste0(c1, c2, collapse = "+")
```

```
[1] "ac+bd"
```

Miscellaneous Functions for Text Manipulation (cont.)

- Using `sep` and `collapse` together:

```
paste(c1, c2, sep = "_", collapse = "+")
```

```
[1] "a_c+b_d"
```

- Concatenation between variable and text strings:

```
ans <- 3
paste0("The value of variable ans is ", ans)
```

```
[1] "The value of variable ans is 3"
```

Factors

- In R, we usually use factor variable types for categorical variables.
- Here, vector `f1` is numeric:

```
f1 <- c(0, 1, 1, 1, 0, 0, 1)
```

```
class(f1)
```

```
[1] "numeric"
```

- We can use the `factor()` function to change `f1`:

```
f1 <- factor(f1)
```

```
class(f1)
```

```
[1] "factor"
```

```
f1
```

```
[1] 0 1 1 1 0 0 1
Levels: 0 1
```

Factors (cont.)

- Now that `f1` is a factor vector, we can utilize its levels for our analysis.
- We can use the `levels()` function to check the levels:

```
levels(f1)
```

```
[1] "0" "1"
```

- `as.factor()` will have the same result as `factor()`. This is preferred because it will provide a quicker return.

```
f2 <- as.factor(c("Female", "Male", "Female", "Unknown", "Female", "
    Male", "Male", "Female"))
f2
```

```
[1] Female  Male    Female  Unknown Female  Male    Male    Female
Levels: Female Male Unknown
```

Tables

```
f2
```

```
[1] Female  Male    Female  Unknown Female  Male    Male    Female
Levels: Female Male Unknown
```

- `table()` will produce a table to show how often each object for a categorical (factor) variable is given:

```
table(f2)
```

```
f2
 Female    Male Unknown
      4       3       1
```

- Small trick: to show relative frequency *(or percentage)*, we can do the following:

```
table(f2) / length(f2)
```

```
f2
 Female    Male Unknown
  0.500   0.375   0.125
```

Tables (cont.)

```
gender <- table(f2) / length(f2) # the result will be a named vector
```

```
gender[1]
```

```
Female
   0.5
```

```
names(gender)
```

```
[1] "Female"  "Male"    "Unknown"
```

- Two-way table using `table()`

```
exam1 <- as.factor(c("A", "B", "F", "D", "B", "C", "A"))
exam2 <- as.factor(c("A", "B", "C", "A", "C", "B", "B"))

table(exam1, exam2) # a two-way table
```

```
     exam2
exam1 A B C
    A 1 1 0
    B 0 1 1
    C 0 1 0
    D 1 0 0
    F 0 0 1
```

File Input/Output

- *Save the four reference files to your class folder. NOTE: for the .txt file, you need to right-click and then select "save link as".*
- There are multiple ways to import & export data, depending on what type of file/database it is and where it is stored.
- **Connecting to text files:** `read.table()` is used to import data from `.txt` files:

```
grade_df <- read.table("R Ref Files/grade.txt", header = TRUE)
```

File Input/Output (cont.)

```
class(grade_df$Name)
```

```
[1] "character"
```

```
class(grade_df$Grade)
```

```
[1] "integer"
```

- Using the `stringsAsFactors` argument *(by default = FALSE)*:

```
grade_df2 <- read.table("R Ref Files/grade.txt", header = TRUE,
    stringsAsFactors = TRUE)
```

```
class(grade_df2$Name)
```

```
[1] "factor"
```

File Input/Output (cont.)

- **Connecting to CSVs:** `read.csv()` is used to import data from `.csv` file:

```
tl_df <- read.csv("R Ref Files/termlife.csv", header = TRUE)

# convert the categorical variables in tl_df into factor variables
tl_df$gender <- as.factor(tl_df$gender)
tl_df$marstat <- as.factor(tl_df$marstat)
tl_df$edu <- as.factor(tl_df$edu)
tl_df$numhh <- as.factor(tl_df$numhh)
```

- **Connecting to Excel:** first, you need to install the `readxl` package. *We'll talk more about R packages in our next class.*
- Then, you can use functions from the package to import your excel data.

File Input/Output (cont.)

```
# Import Excel data and name your data in R as follows:
library(readxl)
# To work properly with the below function, the Excel table
# needs to start in cell A1 of the respective worksheet
excel_data <- read_excel(path="C:/Users/ssaul/Desktop/MEPS_year.xlsx",
                         sheet = "raw_data")
```

- You can also import SAS files and connect to other servers. For example, in many companies, it will be helpful to connect directly to Oracle servers.
- **Exporting data:** `write.table()` and `write.csv()` can be used to export `data.frame` to files:

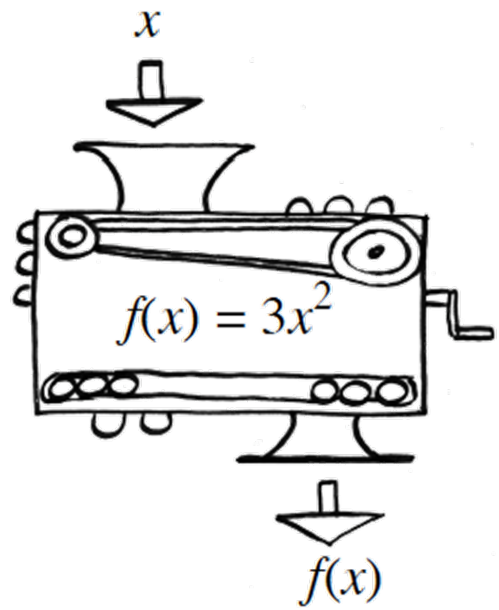
```
write.csv(grade_df, "R Ref Files/grade.csv")
```

Functions, Flow Controls & Loops, and Graphics

Functions

"Abstracting your code into many small functions is key for writing nice R code."

- Remember the concept of functions from school?
- If we plug in 3 for the function $f(x) = 3x^2$ on the right, we will get 27. Easy-peasy.
- Code functions in R work the same way – there are inputs, there are actions that take place with those inputs inside the function, and then there are outputs.



Functions (cont.)

- For example:

```
sum_of_squares <- function(x,y){
  temp <- x^2 + y^2
  return(temp)
}
```

```
sum_of_squares(2,1)
```

```
[1] 5
```

```
sum_of_squares(y=1,x=2)
```

```
[1] 5
```

Functions (cont.)

- Keep in mind that the environment within a function only exists inside that function, unless you assign an output.
  - Therefore, any variables created in a function cannot be used outside of it without you explicitly returning them.
  - Also, anything to be used in the function needs to be included as an input.
- Functions are incredibly useful!
  - If you have code that you want to package so that you can easily use it on many different datasets, lists, etc.
  - If you want to use a function on a new dataset, you only have to update the function call.
- You can think of a function as saved code that performs an action. Super useful!

Functions

Practice

- Create a simple function called `add` with inputs `(x, y)` to return the sum of x and y:

```
add <- function(x, y) {
  temp <- x + y
  return(temp)
}

add(1, 3)
```

```
[1] 4
```

Functions (cont.)

Practice

- Note that you can return your result in a different data structure:

```
add_list <- function(x, y) {
  temp <- x + y
  return(list(vx = x, vy = y, sum = temp))
}
```

```
add_list(1, 3)
```

```
$vx
[1] 1

$vy
[1] 3

$sum
[1] 4
```

Flow controls

If(){else} and ifelse()

```
a <- -1

if (a >= 0) {
      sqrt(a) # sqrt() is used to calculate the square root
    } else if (a >= 1000000) {
      paste0(a, " is too large to calculate.")
    } else {
      paste0(a, " is a negative number.")
}
```

```
[1] "-1 is a negative number."
```

Flow controls (cont.)

If(){else} and ifelse()

- If you only have two cases, then `ifelse()` will be faster:

```
ifelse(a >= 0, sqrt(a), paste0(a, " is a negative number."))
```

```
[1] "-1 is a negative number."
```

- `ifelse()` can also deal with multiple cases by using nested statements, but you need to be careful when applying to different types of data *(i.e. if you have multiple cases, it is safer to use `if(){else}`)*:

```
ifelse(a >= 0, sqrt(a),
       ifelse(a >= 1000000, paste0(a, " is too large to calculate."),
              paste0(a, " is a negative number.")))
```

```
[1] "-1 is a negative number."
```

Flow controls

For loops

- This logic *(and code)* is very comparable to what we were doing with VBA.
- For example, what if we want to calculate the summation of 1 to 10 in a loop?

```
sum <- 0
for (i in 1:10) {
  sum <- sum + i
}
```

```
sum
```

```
[1] 55
```

Flow controls (cont.)

For loops

- To make it more useful, we can include variables in the range of the index:

```
low <- 3
high <- 6

sum2 <- 0
for (i in low:high) {
  sum2 <- sum2 + i
}
```

```
sum2
```

```
[1] 18
```

Flow controls (cont.)

For loops

- Even easier, we can use a pre-defined vector as the range of index:

```
index_seq <- c(2, 6, 10, 22)

sum3 <- 0
for (i in index_seq) {
  sum3 <- sum3 + i
}
```

```
sum3
```

```
[1] 40
```

Flow controls

While loops

```
sum4 <- 0
counter <- 0
while (sum4 <= 10000) {
  counter <- counter + 1
  sum4 <- sum4 + counter
}
counter
```

```
[1] 141
```

```
sum4
```

```
[1] 10011
```

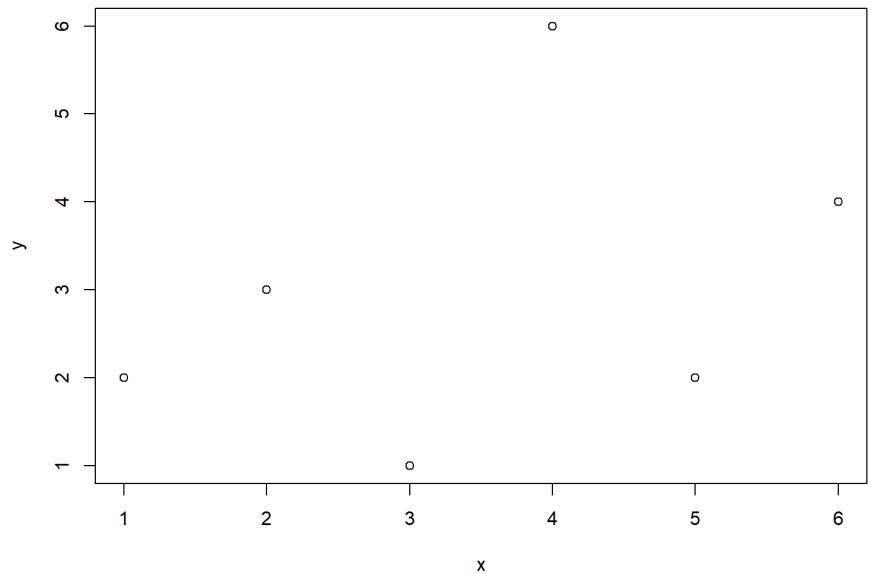
Graphics

Scatter plots

```
x <- 1:6
y <- c(2, 3, 1, 6, 2, 4)
```

- Creating a simple scatter plot:

```
plot(x, y)
```

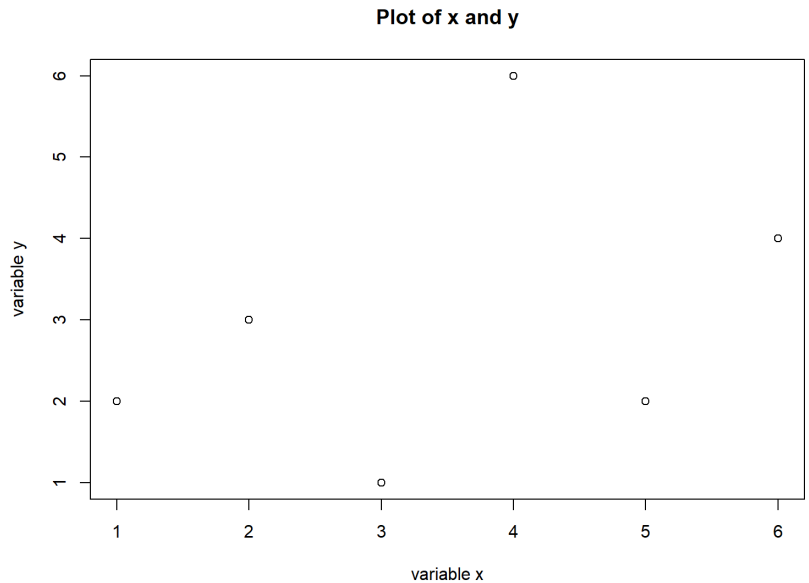


Graphics (cont.)

Scatter plots

- Adding some more elements to the plot:
  - The `main` argument defines the title
  - The `xlab` and `ylab` arguments define the label for the `x` and `y` axis, respectively

```
plot(x, y, main = "Plot of x and y", xlab = "variable x", ylab = "
     variable y")
```

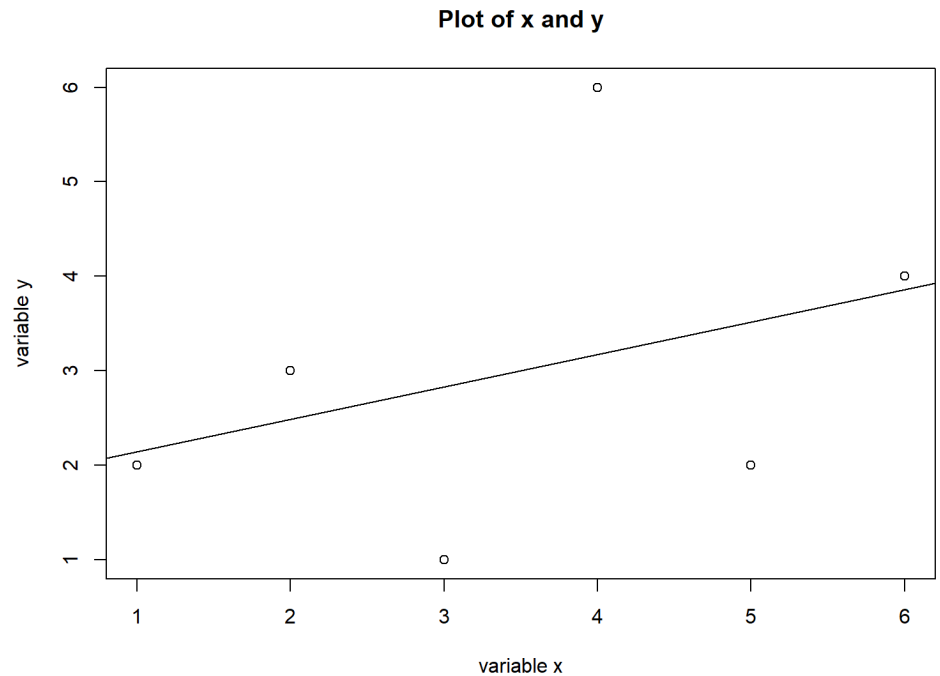


Graphics

Regression lines

- Use `abline()` to add a regression line to our plot. *Note that `lm` stands for "linear model".*

```
plot(x, y, main = "Plot of x and y", xlab = "variable x", ylab = "
     variable y")
abline(lm(y ~ x))
```



Graphics

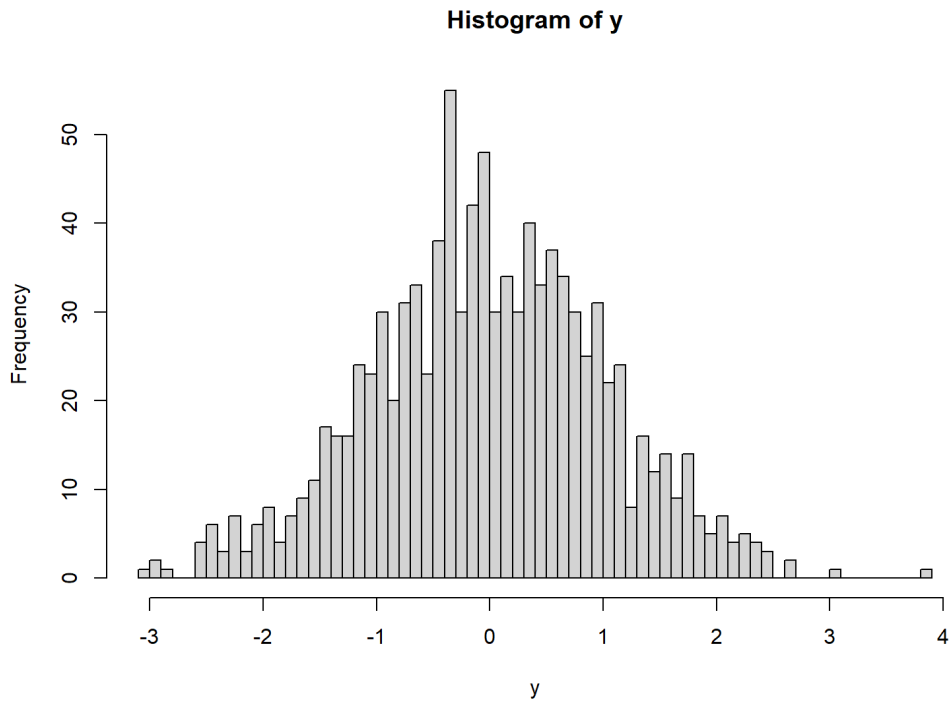
Histograms

- *NOTE: When modeling, we will often want a random selection of the data in order to reflect the best proxy of a true full dataset.*
- *When doing random selection, it is often helpful to use seeding. This will allow for peer review as well as the capability to reproduce the same model, if necessary. This is done by using the* `set.seed()` *function.*
- Let's practice this for the creation of an example histogram, which we can do by using the `hist()` function (next slide).

Graphics (cont.)

Histograms

```
set.seed(1) # fix a set of random numbers
# use rnorm() to generate 1000 random points that follow a standard
    normal distribution
y <- rnorm(1000)
hist(y, breaks = 50) # creates a histogram of y, with 50 bins
```

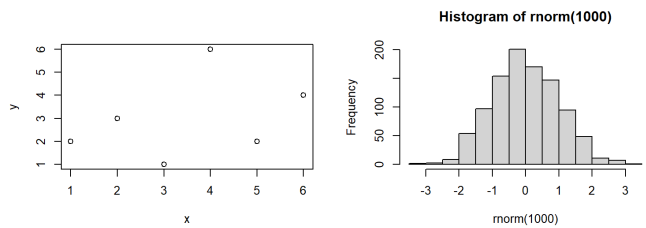


Graphics (cont.)

Multiple graphs in one

- Use `par(mfrow = c(nrow, ncol))` to create multiple graphs in one:

```
x <- 1:6
y <- c(2, 3, 1, 6, 2, 4)
# specify this par() before any graphs, create a 2x2 grid
par(mfrow = c(2, 2))
plot(x, y)
set.seed(0)
hist(rnorm(1000))
par(mfrow = c(1, 1)) # reset back to single graph after you finish
```



Loading and using packages

- R is incredibly useful because of its built-in statistical capabilities, but is also incredibly versatile because of its crowdsourced nature. Professors and professionals from all over the world are constantly inventing new and improved ways to code processes. As a result, this world community creates super convenient things called `packages`. Packages roll together many useful functions for you to access, making coding substantially easier.
- For example, the `stats` package or the `dplyr` package help to aid in data manipulation.

Loading and using packages (cont.)

- *You can take a deep look into what packages contain by using* `vignette()`.
- *Try it out for* `dplyr` *by typing into your Console* `vignette("dplyr")`
- To see the list of installed packages, run `installed.packages()` in your console or go to the "Packages" tab in the Viewer pane.
- When you start an R session, you automatically have several necessary packages loaded to your environment: stats, graphics, utils, and a few others.

Loading and using packages

- Say you want to use the count function from the `dplyr` package, once you've installed it on your machine. You can try the following:

```
count(iris)    # Note: iris is a dataset that already exists in the R
    environment
```

- This doesn't work because there is no `count()` function in base R.
- Usually you do not need to use just one function at a time from a package. To save some keystrokes then, we usually load the packages we need when we start working or at the top of a script, like this:

```
library(dplyr)
count(iris)
```

Loading and using packages (cont.)

- This tells R to attach the `dplyr` package to your environment. Once that happens, all the objects in the package (mostly functions) will also be attached or 'loaded' and made accessible to you without explicit package references in the function call.
- Alternatively, you can supply the package name wherein the count function exists like this:

```
dplyr::count(iris)
```

- This tells R to go to the **libpath**, find the package called `dplyr`, and find the function within there called "count". This is especially useful when you have multiple packages with the same function names.
- For any packages that are not installed already, you can run `install.packages("my_package")`.

Data Cleaning and Processing Techniques

- Run the following code to have a raw dataset called "actuary_raw":

```
x <- 1:8
name <- c("Raymond Li", "", "Peter Smith", NA, "Angela Peterson",
          "Emily Johnson", "David Scott", "Benjamin Lee")
gender <- c("M", "F", "M", "M", "F", "F", "F", "M")
age <- c(25, 23, 22, 50, 30, 42, 29, 36)
exams <- c(10, 3, 0, 4, 6, 7, 5, 9)
Q1 <- c(10, 6, 4, 7, 8, 9, 8, 7)
Q2 <- c(9, 9, 5, 7, 8, 10, 9, 8)
Q3 <- c(9, 7, 5, 8, 10, 10, 7, 8)

actuary_raw <- data.frame(x, name, gender, age, exams, Q1, Q2, Q3)

actuary_raw
```

Data Cleaning and Processing Techniques (cont.)

- The raw dataset `actuary_raw` contains a missing name (`NA`) and an empty name (`""`):

```
  x            name gender age exams Q1 Q2 Q3
1 1      Raymond Li      M  25    10 10  9  9
2 2                      F  23     3  6  9  7
3 3     Peter Smith      M  22     0  4  5  5
4 4            <NA>      M  50     4  7  7  8
5 5 Angela Peterson      F  30     6  8  8 10
6 6   Emily Johnson      F  42     7  9 10 10
7 7     David Scott      F  29     5  8  9  7
8 8    Benjamin Lee      M  36     9  7  8  8
```

Data Cleaning and Processing Techniques (cont.)

- Dealing with missing values:
- Remove observations that contain "NA" *(missing values)* by using the `complete.cases()` function:

```
cc <- complete.cases(actuary_raw)
actuary_new <- actuary_raw[cc, ]

actuary_new
```

```
  x            name gender age exams Q1 Q2 Q3
1 1      Raymond Li      M  25    10 10  9  9
2 2                      F  23     3  6  9  7
3 3     Peter Smith      M  22     0  4  5  5
5 5 Angela Peterson      F  30     6  8  8 10
6 6   Emily Johnson      F  42     7  9 10 10
7 7     David Scott      F  29     5  8  9  7
8 8    Benjamin Lee      M  36     9  7  8  8
```

Data Cleaning and Processing Techniques (cont.)

- Can also use the `na.omit()` function:

```
actuary_new <- na.omit(actuary_raw)
```

- Remove observations that contain "" *("nothing character")* by using subsetting by logical vector:

```
actuary_new$name == ""
```

```
[1] FALSE  TRUE FALSE FALSE FALSE FALSE FALSE
```

```
actuary_new <- actuary_new[!actuary_new$name == "", ]

actuary_new
```

```
  x             name gender age exams Q1 Q2 Q3
1 1       Raymond Li      M  25    10 10  9  9
3 3      Peter Smith      M  22     0  4  5  5
5 5 Angela Peterson      F  30     6  8  8 10
6 6    Emily Johnson      F  42     7  9 10 10
7 7      David Scott      F  29     5  8  9  7
8 8     Benjamin Lee      M  36     9  7  8  8
```

Data Cleaning and Processing Techniques

- Deleting unimportant variables:

```
actuary_new$x <- NULL

actuary_new
```

```
              name gender age exams Q1 Q2 Q3
1       Raymond Li      M  25    10 10  9  9
3      Peter Smith      M  22     0  4  5  5
5 Angela Peterson      F  30     6  8  8 10
6    Emily Johnson      F  42     7  9 10 10
7      David Scott      F  29     5  8  9  7
8     Benjamin Lee      M  36     9  7  8  8
```

Data Cleaning and Processing Techniques (cont.)

- Adding new variables:
- Add a new variable called S, which equals to the average of Q1-Q3:

```
actuary_new$S <- (actuary_new$Q1 + actuary_new$Q2 + actuary_new$Q3) / 3

actuary_new
```

```
             name gender age exams Q1 Q2 Q3        S
1      Raymond Li      M  25    10 10  9  9 9.333333
3     Peter Smith      M  22     0  4  5  5 4.666667
5 Angela Peterson      F  30     6  8  8 10 8.666667
6   Emily Johnson      F  42     7  9 10 10 9.666667
7     David Scott      F  29     5  8  9  7 8.000000
8    Benjamin Lee      M  36     9  7  8  8 7.666667
```

Data Cleaning and Processing Techniques (cont.)

- The averaging formula above is a bit clunky.
- Instead, we can use the `apply(data, margin, function to apply)` function:
  - *MARGIN = 1: apply to rows, MARGIN = 2: apply to columns*

```
actuary_new$S <- apply(actuary_new[, c("Q1", "Q2", "Q3")], MARGIN = 1,
    mean)

# Average of each column
apply(actuary_new[, c("Q1", "Q2", "Q3")], MARGIN = 2, mean)
```

```
      Q1       Q2       Q3
7.666667 8.166667 8.166667
```

Data Cleaning and Processing Techniques (cont.)

- Adding a new variable called `title`, a categorical variable with 3 levels:
  - "FSA" if `exams` $= 10$
  - "ASA" if `exams` are between 7 and 9
  - "student" otherwise

```
actuary_new$title <- ifelse(actuary_new$exams == 10, "FSA",
                            ifelse(actuary_new$exams >= 7 & actuary_new
    $exams <= 9,
                                   "ASA", "student"))

actuary_new
```

```
             name gender age exams Q1 Q2 Q3        S   title
1      Raymond Li      M  25    10 10  9  9 9.333333     FSA
3     Peter Smith      M  22     0  4  5  5 4.666667 student
5 Angela Peterson      F  30     6  8  8 10 8.666667 student
6   Emily Johnson      F  42     7  9 10 10 9.666667     ASA
7     David Scott      F  29     5  8  9  7 8.000000 student
8    Benjamin Lee      M  36     9  7  8  8 7.666667     ASA
```

Data Cleaning and Processing Techniques (cont.)

- We can add new observations by using `rbind()`:

```
new_obs <- c("Bruce Lee", "M", 30, 5, 10, 10, 10, 10, "student")
actuary_new <- rbind(actuary_new, new_obs)
actuary_new
```

```
              name gender age exams Q1 Q2 Q3                 S   title
1       Raymond Li      M  25    10 10  9  9 9.33333333333333     FSA
3      Peter Smith      M  22     0  4  5  5 4.66666666666667 student
5  Angela Peterson      F  30     6  8  8 10 8.66666666666667 student
6    Emily Johnson      F  42     7  9 10 10 9.66666666666667     ASA
7      David Scott      F  29     5  8  9  7                 8 student
8     Benjamin Lee      M  36     9  7  8  8 7.66666666666667     ASA
71       Bruce Lee      M  30     5 10 10 10                10 student
```

Data Cleaning and Processing Techniques

Sorting

- We can use the `order(criteria, decreasing = FALSE)` function to sort your dataframe:

```
actuary_sortbyage <- actuary_new[order(actuary_new$age), ]
actuary_sortbyage
```

```
               name gender age exams Q1 Q2 Q3                 S   title
3      Peter Smith      M  22     0  4  5  5 4.66666666666667 student
1       Raymond Li      M  25    10 10  9  9 9.33333333333333     FSA
7      David Scott      F  29     5  8  9  7                 8 student
5  Angela Peterson      F  30     6  8  8 10 8.66666666666667 student
71       Bruce Lee      M  30     5 10 10 10                10 student
8     Benjamin Lee      M  36     9  7  8  8 7.66666666666667     ASA
6    Emily Johnson      F  42     7  9 10 10 9.66666666666667     ASA
```

Data Cleaning and Processing Techniques (cont.)

Sorting

```
actuary_sortbyagedesc <- actuary_new[order(actuary_new$age, decreasing
    = TRUE), ]
actuary_sortbyagedesc
```

	name	gender	age	exams	Q1	Q2	Q3	S	title
6	Emily Johnson	F	42	7	9	10	10	9.66666666666667	ASA
8	Benjamin Lee	M	36	9	7	8	8	7.66666666666667	ASA
5	Angela Peterson	F	30	6	8	8	10	8.66666666666667	student
71	Bruce Lee	M	30	5	10	10	10	10	student
7	David Scott	F	29	5	8	9	7	8	student
1	Raymond Li	M	25	10	10	9	9	9.33333333333333	FSA
3	Peter Smith	M	22	0	4	5	5	4.66666666666667	student

Data Cleaning and Processing Techniques (cont.)

Sorting

- We can also perform a multi-layer sort.
- Let's sort by gender first, then by S, both in ascending order:

```
actuary_multisort <- actuary_new[order(actuary_new$gender, actuary_new$
    S), ]
actuary_multisort
```

```
              name gender age exams Q1 Q2 Q3                 S   title
7      David Scott      F  29     5  8  9  7                 8 student
5  Angela Peterson      F  30     6  8  8 10 8.66666666666667 student
6    Emily Johnson      F  42     7  9 10 10 9.66666666666667     ASA
71       Bruce Lee      M  30     5 10 10 10                10 student
3      Peter Smith      M  22     0  4  5  5 4.66666666666667 student
8     Benjamin Lee      M  36     9  7  8  8 7.66666666666667     ASA
1       Raymond Li      M  25    10 10  9  9 9.33333333333333     FSA
```

Introduction to Advanced Topics

Why R for Statistical Modeling?

- R is a statistical programming language! Statistics are built-in from its core, which is why it's really good for predictive modeling including but not limited to actuarial applications.
- For example: If you want to get random samples from a particular distribution, there are functions in R to do so:
  - **Normal Distribution**: `rnorm()`
  - **Uniform Distribution**: `runif()`
  - **Poisson Distribution**: `rpois()`
  - **Gamma Distribution**: `rgamma()`
  - **Binomial Distribution**: `rbinom()`

A Simplified Walk-Through of Modeling in R

- We will be using the `caret` package today.
- If you haven't already, please install the `caret` package by entering `install.packages("caret")` in your Console.
- Once you have the `caret` package on your computer, you can call its functions by using the `library` statement:

```
library(caret)
```

- Next, we need to set our working directory so that we can use relative file references.
  - If you have created a `.RProj` file in your directory, you just need to select it by using the dropdown in the top-right corner of the RStudio window.
  - If not, you can use the following code to set your working directory *(yours will be a different path than is shown)*:

```
setwd("C:/Users/ssaul/Desktop/Teaching/ACMA 231")
```

A Simplified Walk-Through of Modeling in R (cont.)

- If you have not already done so, download the `house.csv` file from Canvas and save it in your working directory.
- The `house.csv` data file contains housing values in suburbs of Boston:
  - `x` is an index variable, which should be removed later
  - `medv` is the median value of owner-occupied homes in $1000s
  - `age` is the proportion of owner-occupied units built prior to 1940
  - `lstat` is the lower percent status of the population

A Simplified Walk-Through of Modeling in R (cont.)

- Let's import our data into our R environment:

```
df <- read.csv("R Ref Files/house.csv", header = TRUE)
```

- Taking a look at the first 6 rows of the data:

```
head(df)
```

```
  x medv  age lstat
1 1 24.0 65.2  4.98
2 2 21.6 78.9  9.14
3 3 34.7 61.1  4.03
4 4 33.4 45.8  2.94
5 5 36.2 54.2  5.33
6 6 28.7 58.7  5.21
```

- Let's remove the variable `x` as it is not needed:

```
df$x <- NULL
```

- Let's check to see if the data has any `NA` *(missing values)*. Note that the computer sees TRUE $= 1$ and FALSE $= 0$. Therefore, we can use this fact to test if there are any missing values:

```
nrow(df) - sum(complete.cases(df))
```

```
[1] 0
```

A Simplified Walk-Through of Modeling in R (cont.)

Data Exploration/Processing

- Let's check if the data has any "" *(empty characters)*. *Remember that MARGIN = 2 means that we are aggregating by column.*

```
apply(df == "", 2, sum)
```

```
 medv   age lstat
    0     0     0
```

- Let's take a look at some of the attributes of our data:

```
nrow(df)
```

```
[1] 506
```

```
ncol(df)
```

```
[1] 3
```

A Simplified Walk-Through of Modeling in R (cont.)

Data Exploration/Processing

- Let's take a look at some summary statistics:

```
summary(df)
```

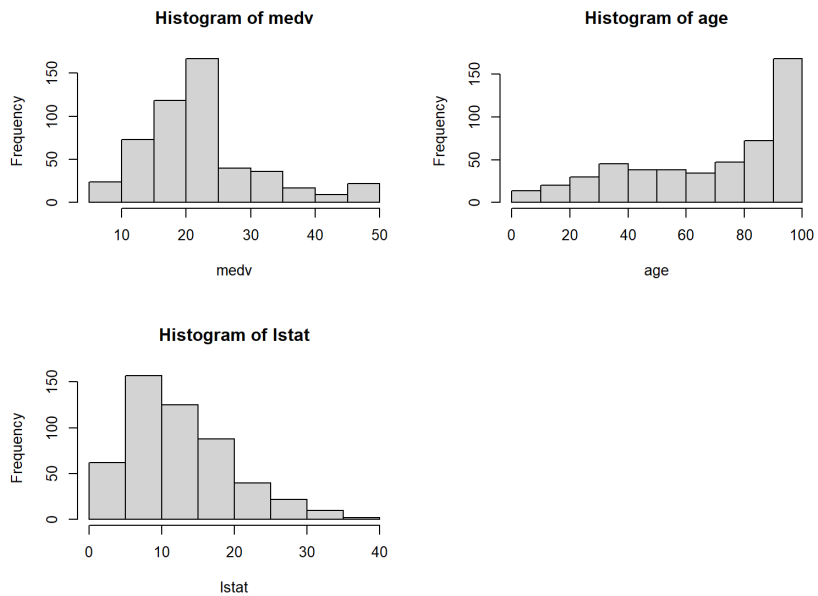
```
      medv            age              lstat
 Min.   : 5.00   Min.   :  2.90   Min.   : 1.73
 1st Qu.:17.02   1st Qu.: 45.02   1st Qu.: 6.95
 Median :21.20   Median : 77.50   Median :11.36
 Mean   :22.53   Mean   : 68.57   Mean   :12.65
 3rd Qu.:25.00   3rd Qu.: 94.08   3rd Qu.:16.95
 Max.   :50.00   Max.   :100.00   Max.   :37.97
```

A Simplified Walk-Through of Modeling in R (cont.)

Data Exploration/Processing

- Let's try out graphical data exploration by doing some histograms:

```
par(mfrow = c(2, 2))
for (i in 1:ncol(df)) {
  hist(df[, i], main = paste0("Histogram of ", names(df)[i]), xlab =
     names(df)[i])
}
par(mfrow = c(1, 1))
```



A Simplified Walk-Through of Modeling in R

- For modeling, since it's impossible to have all of the data from the full real population, we need to work with the data we have. Thus, we want to create a proxy situation for the full, real data.
- We do this by splitting our data into two datasets: `Training` and `Test`.
  - We will use the `Training` data to adjust and build our model *(i.e. "train" our model)* how to interpret, and thus predict, the behavior of the data.
  - We then do a final test of how well this model might work with the real population by using a proxy dataset, our `Test` data.
- The general rule of thumb on how to split the data is a random 80/20 split, 80% going to the `Training` dataset and 20% going to the `Test`.

```
set.seed(0)
randrows <- sample(1:nrow(df), 0.8 * nrow(df))
df_train <- df[randrows, ]
df_test <- df[-randrows, ]
```

A Simplified Walk-Through of Modeling in R

Fitting Linear Regression Models

- We use the `lm(formula, data)` function to fit simple univariate linear models in R.
- In this data, we will use `medv` as our dependent *(target)* variable, and will use `age` & `lstat` and their derivation terms as independent variables *(predictors)*:

```
model_1 <- lm(medv ~ age + lstat, data = df_train)
```

A Simplified Walk-Through of Modeling in R (cont.)

Fitting Linear Regression Models

- Let's take a look at the model summary and its results *(coefficients)*:

```
summary(model_1)
```

```
Call:
lm(formula = medv ~ age + lstat, data = df_train)

Residuals:
    Min      1Q  Median      3Q     Max
-15.927  -3.976  -1.325   1.989  23.245

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept) 33.29776    0.81765  40.724   <2e-16 ***
age          0.03217    0.01368   2.352   0.0191 *
lstat       -1.02408    0.05484 -18.673   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 6.295 on 401 degrees of freedom
Multiple R-squared:  0.5406, Adjusted R-squared:  0.5383
F-statistic: 235.9 on 2 and 401 DF,  p-value: < 2.2e-16
```

A Simplified Walk-Through of Modeling in R (cont.)

Fitting Linear Regression Models

```
model_1$coefficients
```

```
(Intercept)         age       lstat
33.29776447  0.03216801 -1.02408153
```

- Next, let's include an interaction term.
  - Note that `*` in the `lm()` function does not signify multiplication. It means the model contains an interaction term between the two variables (as well as both main effects).
  - *If you want to express multiplication only (the interaction term itself), use* `:`*. For example,* `age:lstat`*.*

A Simplified Walk-Through of Modeling in R (cont.)

Fitting Linear Regression Models

```
model_2 <- lm(medv ~ age * lstat, data = df_train)

summary(model_2)
```

```
Call:
lm(formula = medv ~ age * lstat, data = df_train)

Residuals:
    Min      1Q  Median      3Q     Max
-15.821  -3.991  -1.250   2.025  23.991

Coefficients:
             Estimate Std. Error t value Pr(>|t|)
(Intercept) 34.800861   1.605666  21.674  < 2e-16 ***
age          0.013281   0.022102   0.601    0.548
lstat       -1.213680   0.182743  -6.641 1.02e-10 ***
age:lstat    0.002216   0.002037   1.088    0.277
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 6.294 on 400 degrees of freedom
Multiple R-squared:  0.542,  Adjusted R-squared:  0.5385
F-statistic: 157.8 on 3 and 400 DF,  p-value: < 2.2e-16
```

A Simplified Walk-Through of Modeling in R (cont.)

Fitting Linear Regression Models

- Let's include a squared term. Note that the `^` operator has a different meaning in the `lm()` function, so we need to use `I()` for higher power terms.

```
# I() refers insulate
model_3 <- lm(medv ~ lstat + I(lstat ^ 2), data = df_train)
summary(model_3)
```

```
Call:
lm(formula = medv ~ lstat + I(lstat^2), data = df_train)

Residuals:
     Min       1Q   Median       3Q      Max
-15.2377  -3.8062  -0.5611   2.4278  25.4438

Coefficients:
             Estimate Std. Error t value Pr(>|t|)
(Intercept) 42.715795   0.986307   43.31   <2e-16 ***
lstat       -2.317835   0.140002  -16.56   <2e-16 ***
I(lstat^2)   0.043265   0.004239   10.21   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 5.647 on 401 degrees of freedom
Multiple R-squared:  0.6303, Adjusted R-squared:  0.6284
F-statistic: 341.8 on 2 and 401 DF,  p-value: < 2.2e-16
```

A Simplified Walk-Through of Modeling in R

Fitting Linear Regression Models

- Let's predict using our linear models. We can use the `predict(model, newdata)` function.
- *Note that if we omit the argument `newdata`, then it will use our `Training` data instead of the designated `Test` data.*

```
model_1_pred <- predict(model_1, newdata = df_test)
model_2_pred <- predict(model_2, newdata = df_test)
model_3_pred <- predict(model_3, newdata = df_test)
```

- We would like to use Root Mean Squared Error (RMSE) as our metric of prediction accuracy.
  - You will learn more about RMSE and all of these concepts in STAT 452 (or any equivalent courses), but you can google the formula of RMSE if you are interested.
  - Essentially, the smaller the RMSE, the better the accuracy.

A Simplified Walk-Through of Modeling in R (cont.)

Fitting Linear Regression Models

- We will use the `RMSE(actual value, predicted value)` function from the `caret` package:

```
RMSE(df_test$medv, model_1_pred)
```

```
[1] 5.669556
```

```
RMSE(df_test$medv, model_2_pred)
```

```
[1] 5.581035
```

```
RMSE(df_test$medv, model_3_pred)
```

```
[1] 5.009564
```

A Simplified Walk-Through of Modeling in R

Simulation of Linear Models

- `Y = b0 + b1 * X + e`, i.e. $Y = b_0 + b_1 X + e$, where e is an error term and we assume $e \sim N(0, 2^2)$. Also, $b_0 = 10$ and $b_1 = 5$.

```
# Initialization
n_sim <- 100000
b0 <- 10
b1 <- 5
# make our result reproducible
set.seed(0)
# assume that X ~ N(0, 1)
X <- rnorm(n_sim) # simulation of X
e <- rnorm(n_sim, 0, 2) # simulation of e
# Thus, we can have our simulated Y by
Y <- b0 + b1 * X + e
summary(Y)
```

```
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
-13.474   6.342  10.014   9.999  13.650  34.144
```

A Simplified Walk-Through of Modeling in R (cont.)

Simulation of Linear Models

- We can regress our simulated `Y` against `X`:

```
# We can regress our simulated Y against X
df <- data.frame(cbind(Y, X))
model <- lm(Y ~ X, data = df)
summary(model)
```

```
Call:
lm(formula = Y ~ X, data = df)

Residuals:
    Min      1Q  Median      3Q     Max
-9.5073 -1.3444  0.0036  1.3392  8.5711

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept) 9.991478   0.006306    1584   <2e-16 ***
X           5.008128   0.006300     795   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.994 on 99998 degrees of freedom
Multiple R-squared:  0.8634, Adjusted R-squared:  0.8634
F-statistic: 6.32e+05 on 1 and 99998 DF,  p-value: < 2.2e-16
```

A Simplified Walk-Through of Modeling in R

Simulation of a 95% Confidence Interval

- For a random sample of size n from $N(\mu, \sigma^2)$ with sample mean $\bar{x}$:

$$P\left(\bar{x} - 1.96\frac{\sigma}{\sqrt{n}} \leq \mu \leq \bar{x} + 1.96\frac{\sigma}{\sqrt{n}}\right) = 0.95$$

- Interpretation: If we repeat the CI construction process for a large number of independent random samples, then we should expect that 95% of those realized CIs will include the true mean.

A Simplified Walk-Through of Modeling in R (cont.)

Simulation of a 95% Confidence Interval

```r
# Initialization
n <- 100                # size of random sample we are going to draw in each simulation
n_sim <- 1000000        # number of simulation times
mu <- 5                 # true mean for the normal dist
sigma <- 2              # true sd for the normal dist
count <- rep(NA, n_sim) # create an empty vector to store our result

# Simulation
set.seed(0)

for (i in 1:n_sim) {
  # draw a random sample with size n from the N(5, 2^2)
  x <- rnorm(n, mu, sigma)
  # RHS is a logical test to test whether the true mean is contained in the
  # realized CI constructed by the random sample above or not; if yes,
  # then assign TRUE to count[i], and vice versa.
  count[i] <- (abs(mean(x) - mu) <= qnorm(0.975) * sigma / sqrt(n))
}

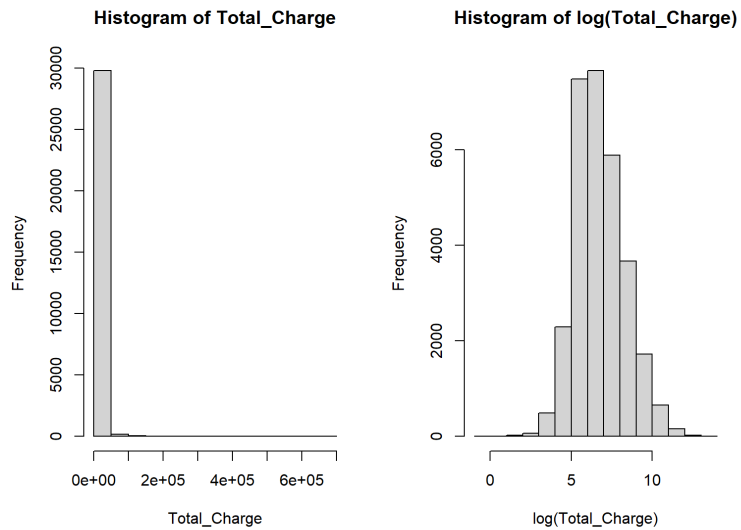
# Finally, we can calculate the average of count to see the approx. percentage
# of how many true means fall into the realized CI in these simulations
mean(count)
```

```
[1] 0.949876
```

Predictive Analytics with R

Data exploration

```
library(readxl)
library(caret)
raw_MEPS <- read_excel(path="MEPS.xlsx",
                       sheet = "raw_data")
par(mfrow=c(1,2))
hist(    raw_MEPS$Total_Charge , main = "Histogram of Total_Charge",
     xlab = "Total_Charge")
hist(log(raw_MEPS$Total_Charge), main = "Histogram of log(Total_Charge)
     ",
     xlab = "log(Total_Charge)")
```



Data exploration (cont.)

```
raw_MEPS$DRSPLTY_M18       <- as.factor(raw_MEPS$DRSPLTY_M18   )
raw_MEPS$MEDPTYPE_M18      <- as.factor(raw_MEPS$MEDPTYPE_M18  )
raw_MEPS$VSTCTGRY          <- as.factor(raw_MEPS$VSTCTGRY      )
raw_MEPS$VSTRELCN_M18      <- as.factor(raw_MEPS$VSTRELCN_M18  )
raw_MEPS$SURGPROC          <- as.factor(raw_MEPS$SURGPROC      )
raw_MEPS$MEDPRESC          <- as.factor(raw_MEPS$MEDPRESC      )
raw_MEPS$TELEHEALTHFLAG    <- as.factor(raw_MEPS$TELEHEALTHFLAG)

summary(raw_MEPS)
```

Data exploration (cont.)

```
  DUPERSID            EVNTIDX              OPDATEYR       OPDATEMM
Length:30079       Length:30079       Min.   :2019   Min.   :-8.000
Class :character   Class :character   1st Qu.:2019   1st Qu.: 4.000
Mode  :character   Mode  :character   Median :2020   Median : 7.000
                                      Mean   :2020   Mean   : 6.661
                                      3rd Qu.:2021   3rd Qu.:10.000
                                      Max.   :2022   Max.   :12.000
 DRSPLTY_M18     MEDPTYPE_M18      VSTCTGRY     VSTRELCN_M18 SURGPROC
-1     :16162   -1     :13917   2      :15542   -8:  143     -8:  188
8      : 1527   15     : 6235   1      : 4469   -7:   32     -7:   29
18     : 1101   8      : 3381   5      : 4107   1 :25153     -1: 2290
20     :  912   10     : 3147   91     : 2581   2 : 4751     1 : 2612
28     :  891   91     :  991   4      : 1453                2 :24960
91     :  820   -8     :  507   6      :  831
(Other): 8666   (Other): 1901   (Other): 1096
MEDPRESC    TELEHEALTHFLAG  Total_Charge
-15: 2247   -15:15652      Min.   :     0.7
-8 :  303   1  : 2290      1st Qu.:   289.0
-7 :   32   2  :12137      Median :   744.3
1  : 2507                  Mean   :  3327.5
2  :24990                  3rd Qu.:  2325.2
                           Max.   :665057.6
```

Data exploration (cont.)

- See the MEPS codebook (meps.ahrq.gov/.../h239fcb.pdf) for the detailed description of each of the covariates.
- One can also check that the response variable (Total_Charge) is strictly positive and continuous from the summary statistics.
- Lastly, we also split the dataset into two parts, for training and out-of-sample validation purposes:

```
set.seed(0)
randrows <- sample(1:nrow(raw_MEPS), 0.8 * nrow(raw_MEPS))
raw_MEPS_train <- raw_MEPS[ randrows, ]
raw_MEPS_test  <- raw_MEPS[-randrows, ]
```

Issues with the dataset

- It is very likely that the dataset that you need to analyze in reality is quite messy, which requires a careful examination and pre-processing if needed. Let us explore existing issues with the given dataset as follows.

1. Possible associations among the covariates (which is called 'co-linearity' in Statistics)

- Consider the descriptions of the following covariates:
  - `DRSPLTY_M18`: the specialty code of the doctor (only available if the outpatient visit was with a doctor),
  - `MEDPTYPE_M18`: type of medical practitioner for the visit (if -1, then it means a doctor).
- You can recognize that `DRSPLTY_M18` and `MEDPTYPE_M18` are somewhat conflicting so that their (negative) association will be high, which requires more careful analysis.

Issues with the dataset (cont.)

2. Existence of too many categories for a covariate

- If you look at the definition of `DRSPLTY_M18` from the codebook, you can check that there are at least 16 categories of medical practitioners.
- So, if you are trying to fit a linear model using such a variable without any pre-processing, the fitted model could be neither interpretable nor provide acceptable prediction performance.

3. Missing values

- You can see that there are some missing values in each of the covariates. We need to think about whether we would just remove all records with missing values, or do something with them.
- Further, we also need to decide whether all missing values are equally treated or not [REFUSED (-7) vs DK (-8)].

Available methods

As such, basically you have two choices here.

- Use a relatively simpler model that could require 'extensive' data pre-processing.
  - e.g. Generalized Linear Models (GLMs)
- Use a more complex model that requires less data pre-processing but comes with less interpretability.
  - e.g. Classification and Regression Trees (CART), Random forest, Neural network, to name a few...

GLMs

- Suppose Y is the response variable (or target variable) that we want to analyze and/or predict, and $\mathbf{x}$ is the covariates (or independent variables).
- In GLMs, we assume a certain relationship between $\mathbb{E}[Y|\mathbf{x}]$ and $\mathbf{x}$ as follows:

$$\mathbb{E}[Y|\mathbf{x}] = g(\mathbf{x}\beta),$$

  where the function g depends on your choice.
- Generally, we assume $g(z) = z$ (identity function) if Y seems to follow a normal distribution, and assume $g(z) = \exp(z)$ if Y seems to follow a gamma or Poisson distribution.
- As our response variable `Total_Charge` is strictly positive and continuous, we will use a gamma distribution for the GLM (in short, gamma GLM) so that $g(z) = \exp(z)$.
- See Chapter 4.6 of 'An Introduction to Statistical Learning' (ISL) textbook (https://www.statlearning.com/) for more explanations on GLMs.

Data pre-processing for a gamma GLM

Here are some modifications that I made on the available covariates **x** for better interpretability and less dimension:

- `DRSPLTY_M18` was converted to `Doctor_Type` (6 or 8: Family practitioners, -1: Non-doctor, others: Specialists)
- I decided not to consider distinctions among the non-doctor practitioners, so `MEDPTYPE_M18` was removed.
- `VSTCTGRY` was converted to `Care_Category` (2: Diagnosis or Treatment, or Others)
- For `VSTRELCN_M18`, `SURGPROC` and `MEDPRESC`, I treated missing values as NO so that there are only 2 categories (YES or NO).
- For `Telehealth`, I treated missing values as MISSING so that there are 3 categories (YES, NO, or MISSING) because it was completely missing for the observations from year 2019 (pre-COVID).

Note that data pre-processing is an open question so there is usually no 'Right Answer'. You just use your own insight and prior knowledge about the domain.

Fitting a gamma GLM

```
refined_MEPS <- read_excel(path="MEPS.xlsx",
                           sheet = "refined_data")
refined_MEPS_train <- refined_MEPS[ randrows, ]
refined_MEPS_test  <- refined_MEPS[-randrows, ]

fit.glm <- glm(Total_Charge ~ Doctor_Type+Care_Category+Special_Cond+
     Surgery+
                 Prescription+Telehealth,
               family="Gamma"(link='log'), data = refined_MEPS_train)

summary(fit.glm)
```

Fitting a gamma GLM (cont.)

```
Call:
glm(formula = Total_Charge ~ Doctor_Type + Care_Category + Special_Cond +
    Surgery + Prescription + Telehealth, family = Gamma(link = "log"),
    data = refined_MEPS_train)

Coefficients:
                      Estimate Std. Error t value Pr(>|t|)
(Intercept)            7.49106    0.08116  92.295  < 2e-16 ***
Doctor_TypeNon-doctor  0.11441    0.06941   1.648  0.09931 .
Doctor_TypeSpecialist  0.51381    0.07071   7.266 3.82e-13 ***
Care_CategoryOthers   -0.15636    0.03527  -4.433 9.33e-06 ***
Special_CondYes        0.17841    0.04725   3.776  0.00016 ***
SurgeryYes             1.71024    0.06425  26.619  < 2e-16 ***
PrescriptionYes       -0.10401    0.06261  -1.661  0.09668 .
TelehealthUnknown     -0.17348    0.03525  -4.921 8.65e-07 ***
TelehealthYes         -0.87512    0.06847 -12.782  < 2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for Gamma family taken to be 6.760725)

    Null deviance: 64560  on 24062  degrees of freedom
Residual deviance: 50122  on 24054  degrees of freedom
AIC: 419305

Number of Fisher Scoring iterations: 7
```

Classification and Regression Trees (CART)

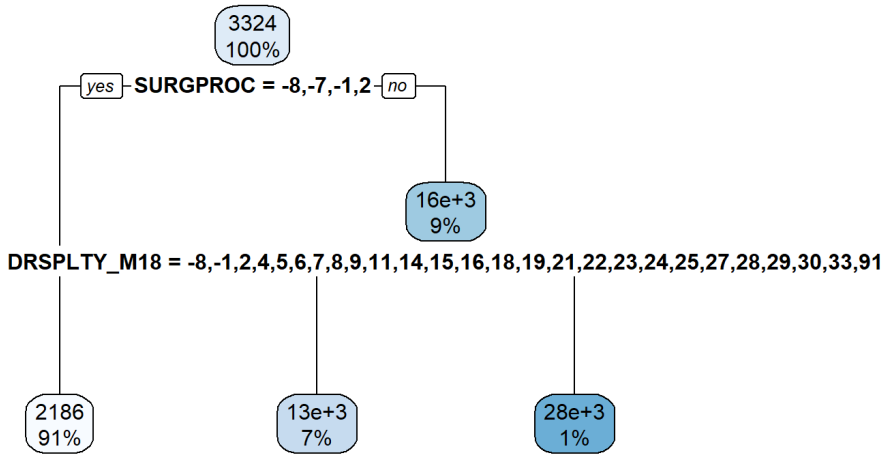
- CART is a decision rule that assigns a value of Y depending on $\mathbf{x}$. You can see that it indeed looks like a 'tree'.

```
library(rpart)
library(rpart.plot)

load("fits.RData")
# fit.tree = rpart(Total_Charge ~ DRSPLTY_M18 + MEDPTYPE_M18 + VSTCTGRY
    +
#                    VSTRELCN_M18 + SURGPROC  + MEDPRESC +
    TELEHEALTHFLAG,
#                  data=raw_MEPS_train)
# it takes a bit more time to fit a CART than a GLM, so I just loaded
# the fitted CART from a saved Rdata file.

rpart.plot(fit.tree)
```

Classification and Regression Trees (CART) (cont.)



- Sometimes the tree could be too messy so it needs 'pruning' for a better performance.
- See Chapter 8.1 of the ISL textbook (https://www.statlearning.com/) for more explanations on CARTs.

Random Forest

- Recall that prediction from a CART model depends on a single tree. So you may want to grow more 'trees' and use all of them to make a decision.
- Random forest is such a concept: grow multiple trees to create a 'forest', and make a decision from the consensus of the forest.
- Note that we do not want to grow identical trees 1000 times to build our forest – so when you train a random forest, a random selection is applied to your dataset so only a part of the dataset is used each time to train each tree.
- Illustration: random forest figure (ResearchGate)

Random Forest (cont.)

```
library(randomForest)
#fit.rf <- randomForest(Total_Charge ~ DRSPLTY_M18 + MEDPTYPE_M18 +
     VSTCTGRY +
#                           VSTRELCN_M18 + SURGPROC  + MEDPRESC +
     TELEHEALTHFLAG,
#                          data=raw_MEPS_train)
# it takes more time to fit a random forest than a GLM, so I just loaded
# the fitted random forest from a saved Rdata file.
```

- See Chapters 8.2.1–8.2.2 of the ISL textbook (https://www.statlearning.com/) for more explanations on random forests.

Neural network

- It is a way more complicated model (sometimes very useful) to find non-linear and complex relationships among the covariates $\mathbf{x}$ to predict Y.
- In order to fit a neural network model, you need a (better to be normalized or standardized) numeric matrix, so it requires a process called 'one-hot encoding' (see https://www.geeksforgeeks.org/ml-one-hot-encoding/).

```
rawonehot_MEPS <- as.data.frame(model.matrix(OPDATEMM  ~.-1 , data=raw_
    MEPS[,4:12]))
onehotnames    <- colnames(rawonehot_MEPS)
colnames(rawonehot_MEPS) <- c(paste0("col", 1: (ncol(rawonehot_MEPS)-1)
    ),
                              onehotnames[ncol(rawonehot_MEPS)])
rawonehot_MEPS_train <- rawonehot_MEPS[ randrows,]
rawonehot_MEPS_test  <- rawonehot_MEPS[-randrows,]

head(rawonehot_MEPS)
```

Neural network (cont.)

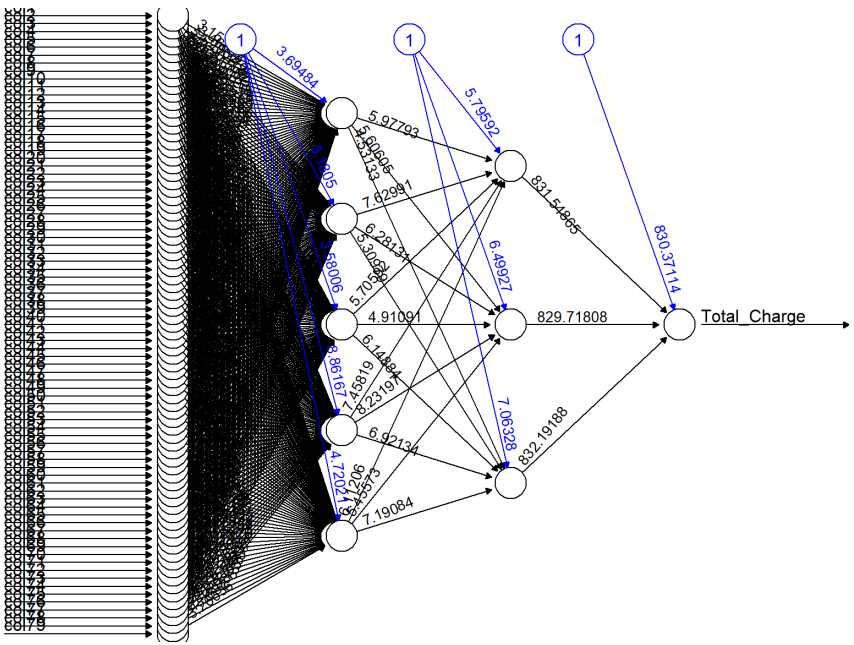
```
  col1 col2 col3 col4 col5 col6 col7 col8 col9 col10 col11 col12 col13 col14
1    0    0    1    0    0    0    0    0    0     0     0     0     0     0
2    0    0    1    0    0    0    0    0    0     0     0     0     0     0
3    0    0    1    0    0    0    0    0    0     0     0     0     0     0
4    0    0    1    0    0    0    0    0    0     0     0     0     0     0
5    0    0    1    0    0    0    0    0    0     0     0     0     0     0
6    0    0    0    0    0    0    0    0    0     1     0     0     0     0
...
  col67 col68 col69 col70 col71 col72 col73 col74 col75 col76 col77 col78 col79
1     0     1     0     0     0     0     1     0     0     0     1     0     0
2     0     1     0     0     0     0     1     0     0     0     1     0     0
3     0     1     0     0     0     0     1     0     0     0     1     0     0
4     0     1     0     0     0     0     1     0     0     0     1     0     0
5     0     1     0     0     0     0     1     0     0     0     1     0     0
6     0     1     0     0     0     1     0     0     0     0     1     0     1
  Total_Charge
1      1209.00
2      5912.30
3      5912.30
4       897.00
5       990.57
6      6407.50
```

- The categorical covariates become 79 dummy (0/1) columns `col1`–`col79`, followed by the response `Total_Charge` (columns `col15`–`col66` omitted above).

Neural network (cont.)

```
library(neuralnet)
#fit.nn <- neuralnet(Total_Charge ~ ., rawonehot_MEPS_train,
#                    hidden=c(5,3),linear.output = TRUE)
# it takes MUCH MORE time to fit a neural network than a GLM, so I just
# loaded the fitted neural network from a saved Rdata file.

plot(fit.nn,rep = "best")
```



- Here is a plot for the fitted neural network with the MEPS data.
- See Chapters 10.1–10.2 of the ISL textbook (https://www.statlearning.com/) for more explanations on neural networks.

Prediction performance

- Lastly, we compare the prediction performance of the four fitted models with the MEPS dataset as follows.

```
pred.glm  <- exp(predict(fit.glm, refined_MEPS_test))
pred.tree <- predict(fit.tree, raw_MEPS_test)
pred.rf   <- predict(fit.rf, raw_MEPS_test)
pred.nn   <- compute(fit.nn, rawonehot_MEPS_test)$net.result

valtable <- rbind(
c(
RMSE(pred.glm, raw_MEPS_test$Total_Charge) ,
RMSE(pred.tree, raw_MEPS_test$Total_Charge),
RMSE(pred.rf, raw_MEPS_test$Total_Charge),
RMSE(pred.nn, raw_MEPS_test$Total_Charge)),
c(
  MAE(pred.glm, raw_MEPS_test$Total_Charge) ,
  MAE(pred.tree, raw_MEPS_test$Total_Charge),
  MAE(pred.rf, raw_MEPS_test$Total_Charge),
  MAE(pred.nn, raw_MEPS_test$Total_Charge)))

colnames(valtable) <- c("GLM", "CART", "RF", "NN")
rownames(valtable) <- c("RMSE", "MAE")

valtable
```

```
            GLM       CART         RF         NN
RMSE 12053.085 12050.383 12063.034 12597.416
MAE   3604.094  3597.488  3457.745  4073.573
```

Prediction performance (cont.)

- As you can see, it is somewhat unclear which model would be the most preferred as RMSE and MAE provide different conclusions (RF is most favored with MAE, whereas GLM or CART is better with RMSE).
- So in this case another judgement is needed based on the insight and preference of the modeler.
- (Personally, I would prefer GLM as it has the best interpretability, better RMSE than RF, and comparable RMSE and MAE to the CART – but it is my choice rather than a right answer.)